

FORENIX-OSS · V0.1 · 2026-05

forenix-oss

OSINT × Forensics — a step-by-step guide for analysts, examiners, and team leads.

Author: Ali Murtaza Bhutto · **Repository:** github.com/thunderstornX/forenix-oss · **License:** MIT

forenix-oss — User Manual

Version 0.1 · 2026-05

Welcome to forenix-oss. This manual is for the analyst, the forensic examiner, and the team lead who'll be using the platform day to day. We assume zero prior knowledge — by the end of this document you'll understand every feature, the workflow it serves, and the keystrokes that make it fast.

The platform is screenshot-driven. Every figure in this manual is a **real** render against a live demo run by hosted LLMs. Nothing is mocked-up in Figma.

Table of contents

1. Introduction
 2. Before you start
 3. Your first launch
 4. The workspace, in 60 seconds
 5. Workflow 1 — Your first OSINT investigation
 6. Workflow 2 — Bridge findings to a forensic case
 7. Workflow 3 — Evidence + chain of custody
 8. Workflow 4 — Verify the audit chain
 9. Workflow 5 — Generate and read reports
 10. Feature reference — every view
 11. Working with the AI adapters
 12. Keyboard shortcuts + command palette
 13. Tips for new analysts
 14. Troubleshooting
 15. Frequently asked questions
 16. Glossary
-

1. Introduction

1.1 What is forenix-oss

forenix-oss is an open-source platform that fuses two workflows that have always lived in separate tools:

- **OSINT (Open-Source Intelligence)** — discovering, organising and reasoning about a target using publicly-available information.
- **Digital forensics** — taking material that may be entered into evidence, hashing it, tracking every change, and proving the custody chain hasn't been tampered with.

Today, an investigator uses one set of tools to do the first half and a completely different set to do the second half. The handoff is manual. The audit trail is artisanal. forenix-oss makes both halves a single, auditable workflow.

1.2 Who this manual is for

- The **analyst** running OSINT pipelines and capturing findings.
- The **forensic examiner** managing evidence and merge-request reviews.
- The **team lead** auditing the chain of custody before hand-off.
- The **operator** deploying the platform and watching its health.

If you're an engineer extending the platform, see `docs/03-SDS.md` , `docs/06-ARCHITECTURE.md` and `docs/08-API.md` instead.

1.3 What you'll learn

By the end of this manual you'll be able to:

1. Run an end-to-end OSINT pipeline against a target.
2. Open a forensic case from any finding and bring its evidence under chain of custody.
3. Verify the integrity of the audit chain yourself.
4. Use every secondary view (network graph, monitors, AI lab, reports, ...) and know what problem each one solves.
5. Switch between AI adapters (mock, NVIDIA, OpenRouter, ...) and understand the tradeoffs.

2. Before you start

2.1 System requirements

- **OS:** Linux, macOS, or Windows-WSL2.
- **Runtime:** Bun 1.3+ or Node.js 20+.
- **Browser:** any modern Chromium-based browser (Chrome, Edge, Brave, Arc) or recent Firefox / Safari.
- **Optional, for hosted LLMs:** an OpenRouter API key (<https://openrouter.ai>), a NVIDIA NIM API key (<https://build.nvidia.com>), or a local Ollama install.

2.2 Where to get help

- This manual.
- The feature catalogue: `docs/FEATURES.md` .
- The operational runbook: `docs/09-RUNBOOK.md` .
- The GitHub issues tracker on <https://github.com/thunderstornX/forenix-oss>.

2.3 The demo dataset

forenix-oss ships with a seed script that creates a complete demo workspace in one command:

- **3 users** — `Admin` , `Jay Investigator` , `Sam Analyst` .

- **2 investigations:**

- INV-2025-019 – Northwind Holdings (status: `complete` , bridged to `CASE-2025-007` – Operation Sandstone).
- INV-2025-020 – Mira Volkov (status: `running` / `complete` after the live demo run).
- **1 seeded case** with 3 evidence items, 2 branches, and 6 commits — plus a second case auto-created when the live demo bridges the second investigation.
- A valid 9-row hash-chained audit log to start with; the chain grows naturally as you operate.

Every screenshot in this manual was taken **after** running the live demo, so the numbers and findings you see are the real output of hosted Large Language Models. Your numbers will look slightly different — the model returns slightly different findings on every run, by design.

3. Your first launch

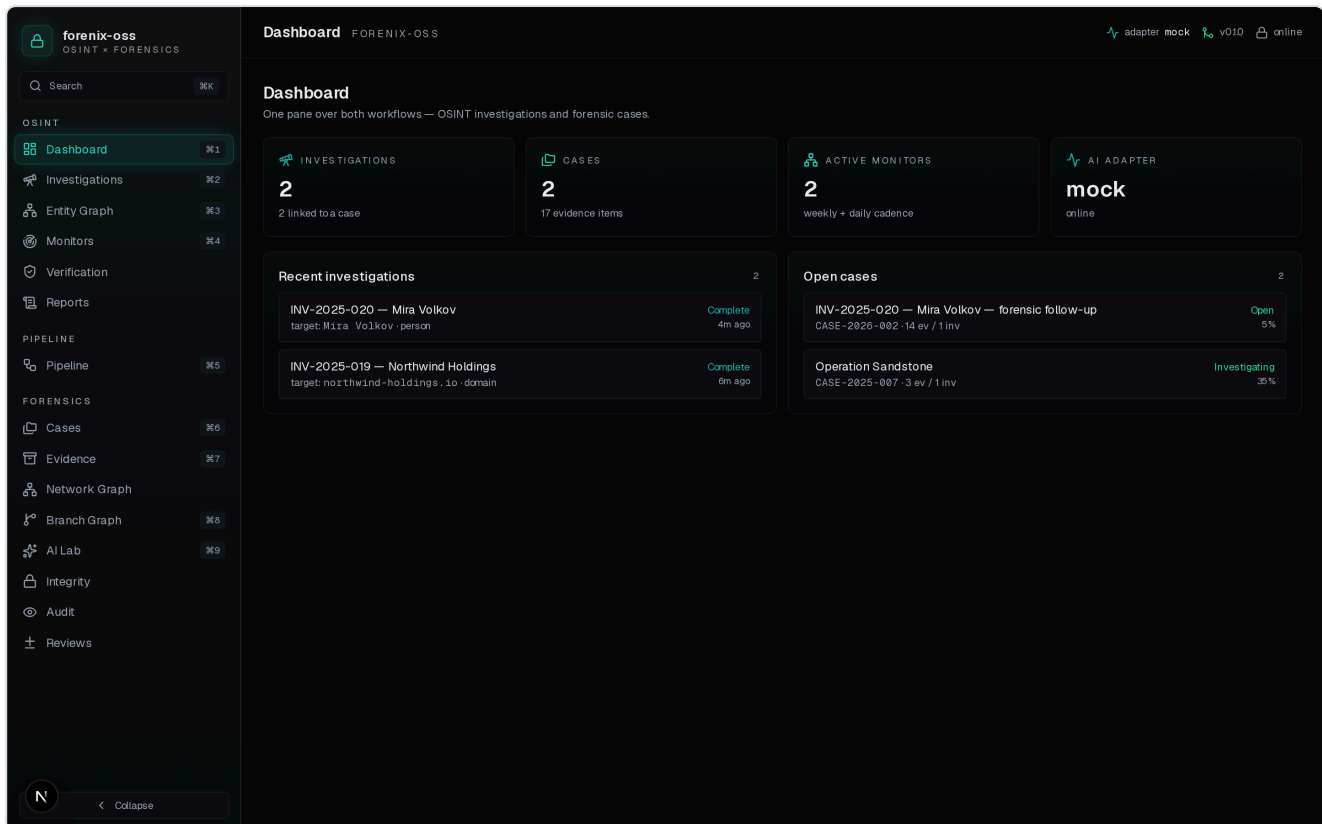
3.1 Install + seed

```
git clone https://github.com/thunderstornX/forenix-oss
cd forenix-oss
bun install
cp .env.example .env
bun run db:push
bun run db:seed
bun run dev
```

Open <http://localhost:3000>. (The screenshots in this manual were taken at port 3737 — the port is configurable.)

3.2 First screen

The landing view is the Dashboard. It pulls live counts from the database and renders both workflows on one pane.



What you're looking at:

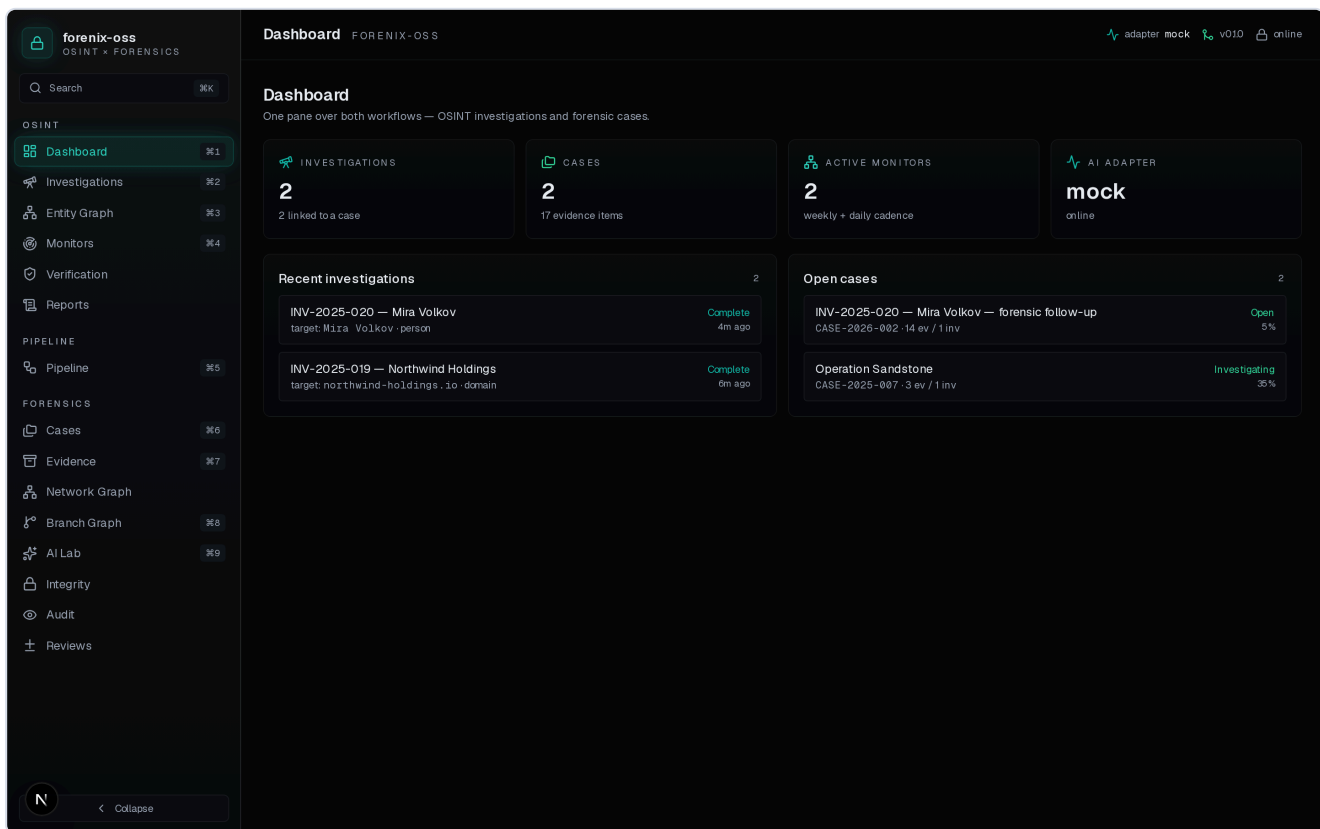
- **Sidebar (left).** Three sections: OSINT (Argus heritage), Pipeline (the bridge view), Forensics (ForenX heritage). The active view is highlighted in teal with a soft glow.
- **Top bar (centre).** The current view's name + the active AI adapter chip + the version + the online status.
- **Stat cards.** Investigations, Cases, Active monitors, AI adapter. Each is wired to a live query — refresh and the numbers update.
- **Recent investigations + Open cases.** Sortable preview rows pulled straight from the database.

Click any row in *Recent investigations* or *Open cases* to drill straight into the detail panel.

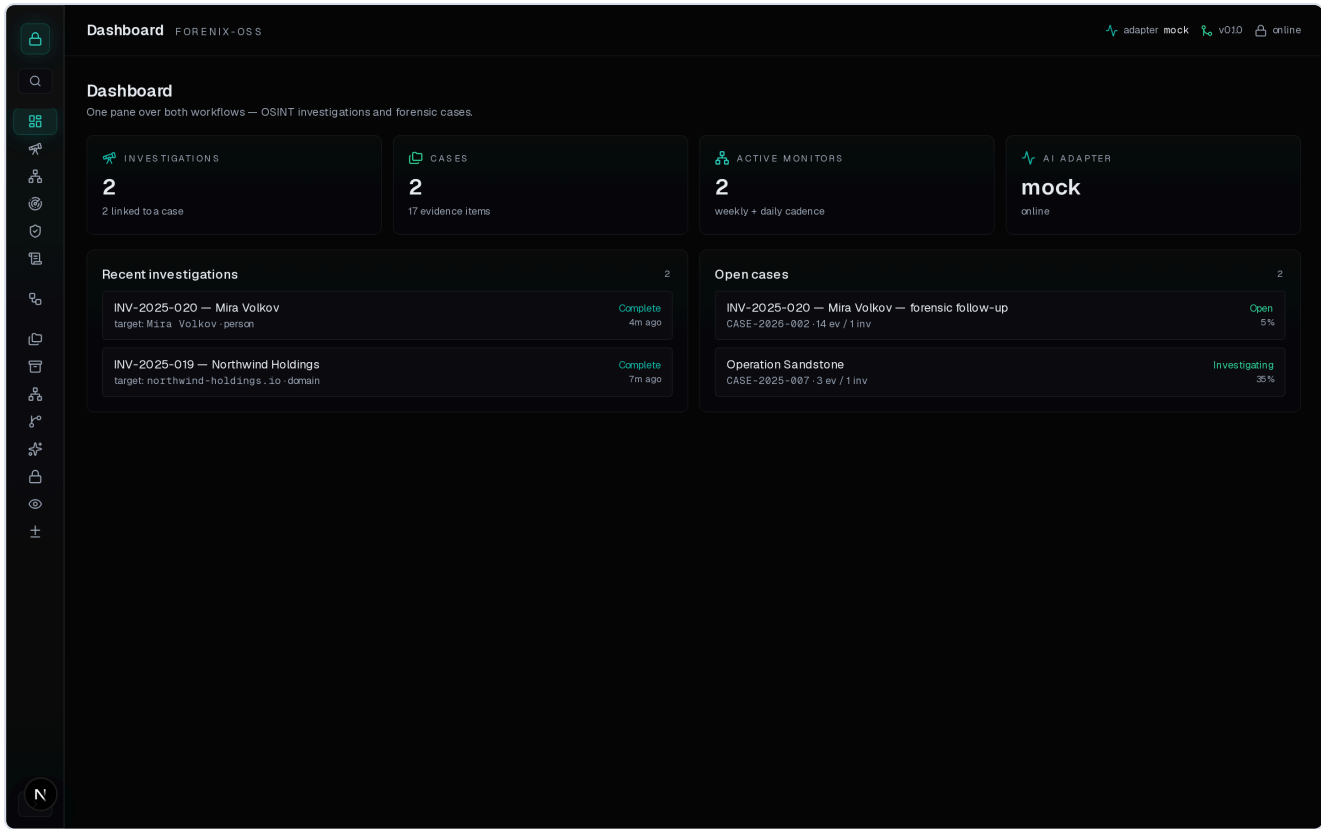
4. The workspace, in 60 seconds

The whole platform is one page (/) with a sidebar-driven view router. Switching views is **instant** — no route load, no re-fetch unless you ask.

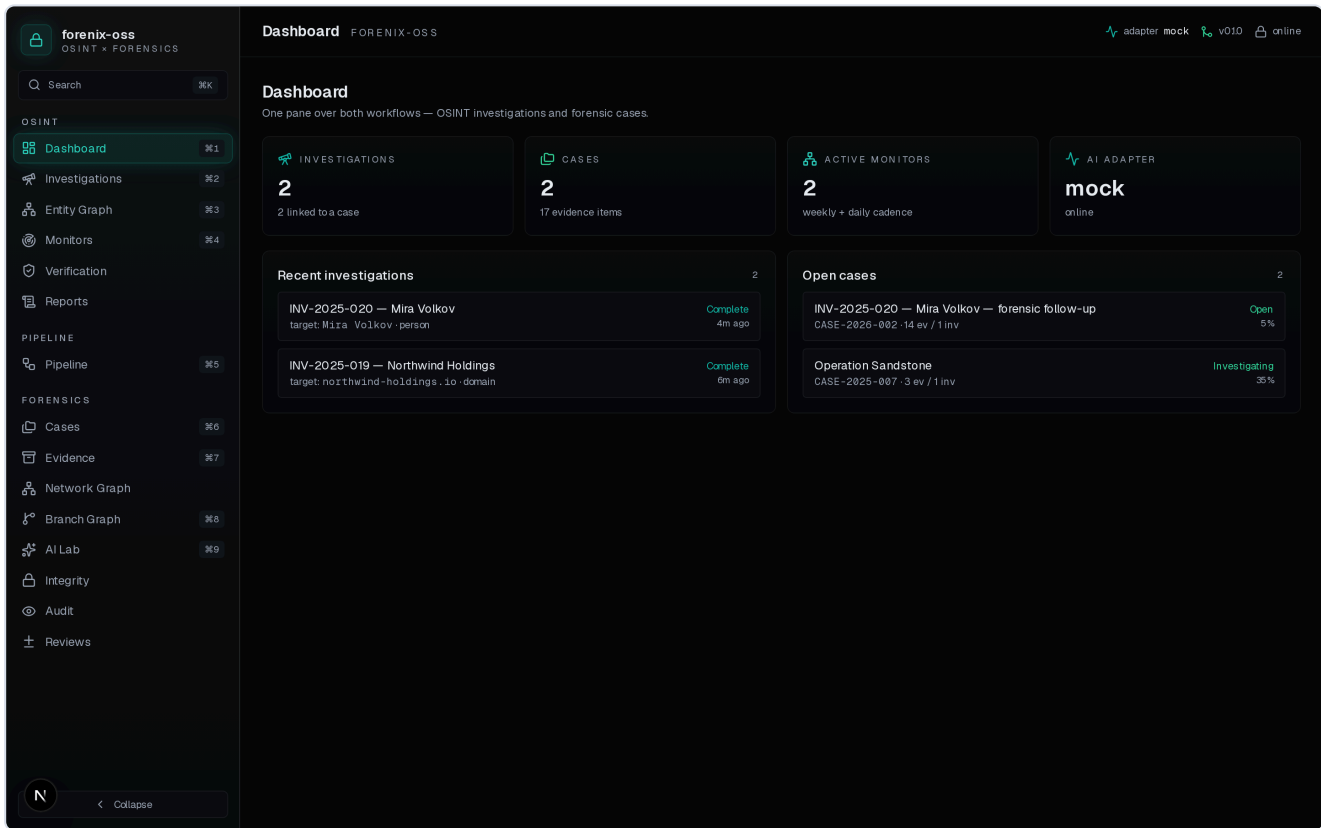
4.1 Sidebar



- **Brand** (top-left) — the platform name + the "OSINT × Forensics" tagline.
- **Search** button — opens the global command palette (⌘K).
- **OSINT section** — Dashboard, Investigations, Entity Graph, Monitors, Verification, Reports.
- **Pipeline section** — the bridge runner.
- **Forensics section** — Cases, Evidence, Network Graph, Branch Graph, AI Lab, Integrity, Audit, Reviews.
- **Collapse toggle** (bottom-left) — slim mode with just icons.



4.2 Top bar



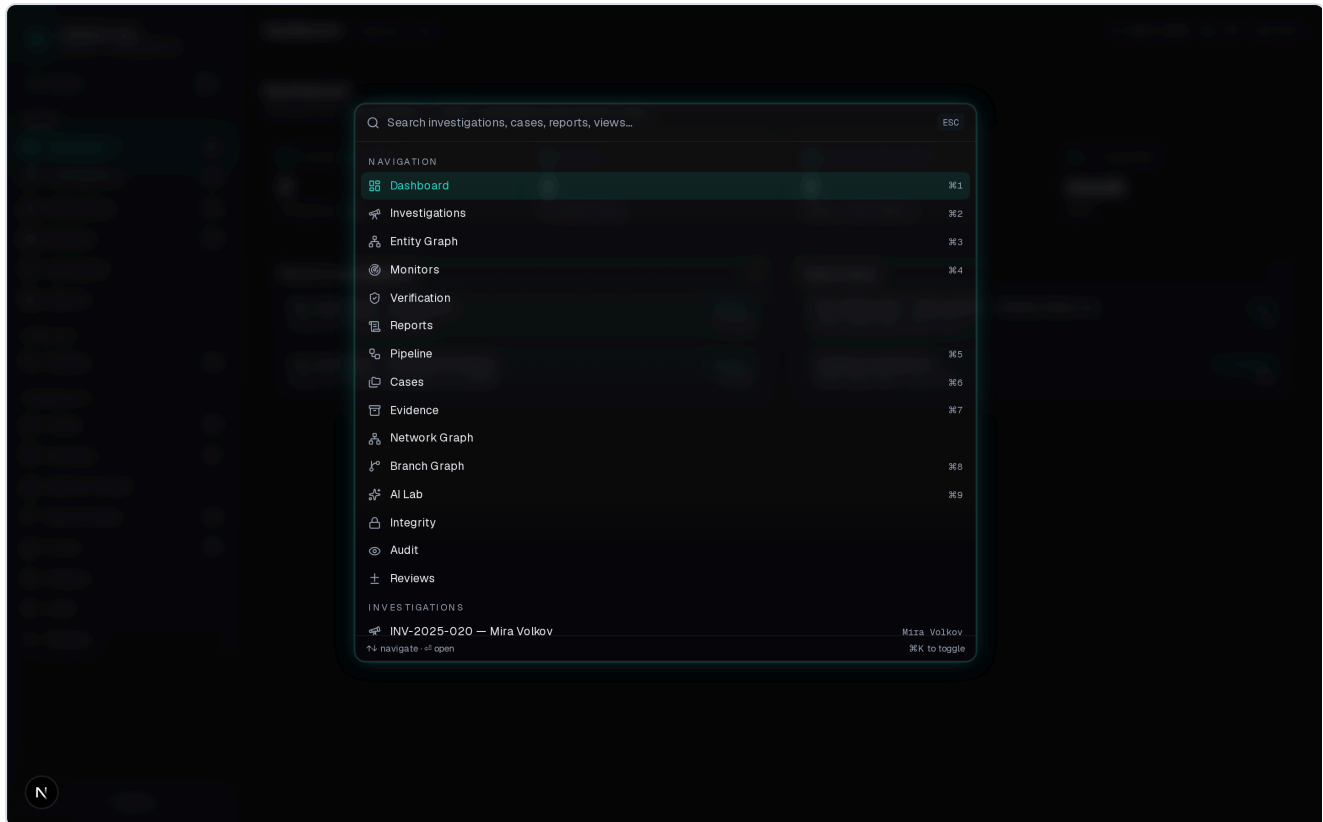
The top bar always shows:

- the **active view's title** (left);

- the **active AI adapter** (right) — useful when you've switched to a real model and want to verify the chip says `nvidia` or `openrouter` ;
- the **version** of the platform;
- the **online** status — green when the database + adapter are reachable.

4.3 Command palette (⌘K)

Press **⌘K** (or **Ctrl+K** on Linux/Windows) anywhere. The palette opens.



Type a few letters of any:

- nav item (e.g. `audit` , `entity`),
- investigation title,
- case number,
- report title.

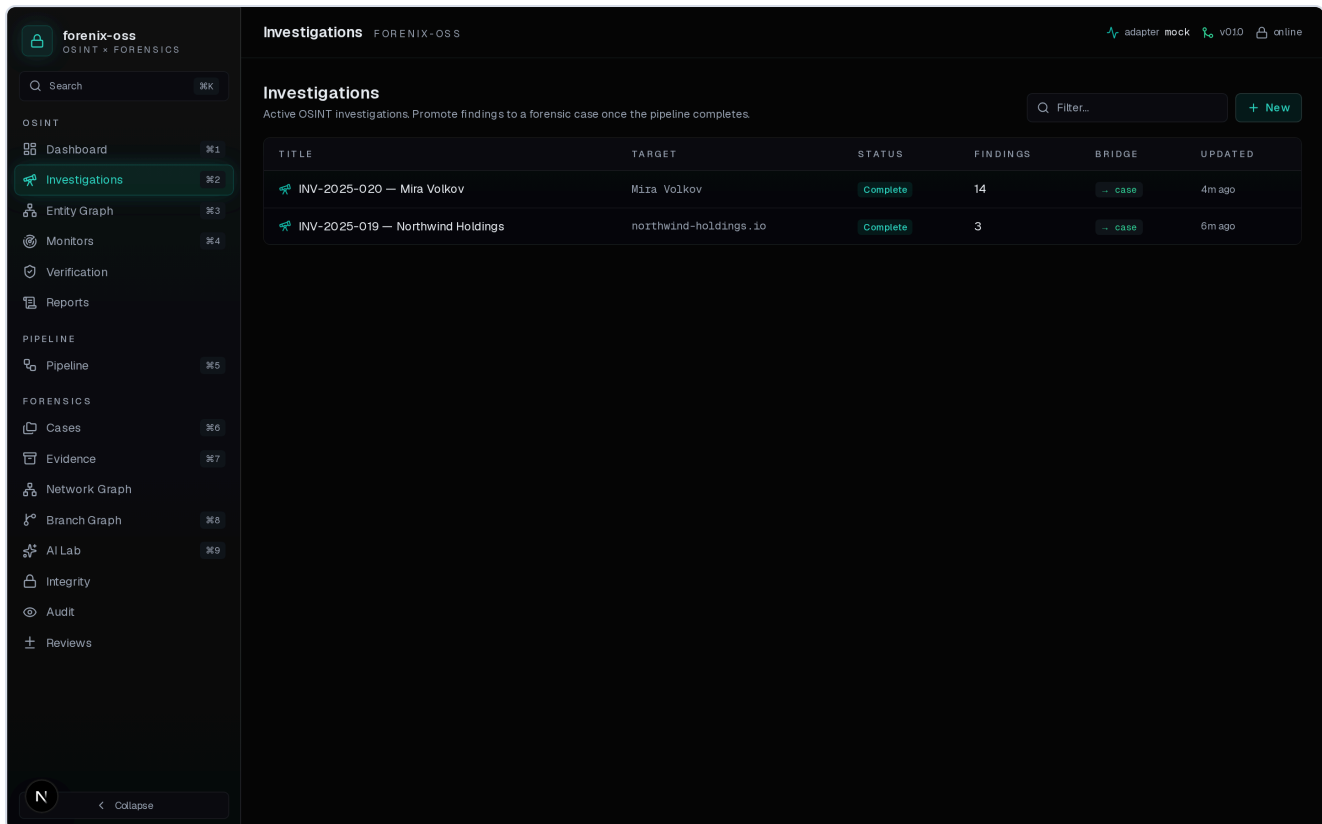
Use the arrow keys to navigate, Enter to open. Esc to close. This is the fastest way to move around the platform once your dataset gets non-trivial.

5. Workflow 1 — Your first OSINT investigation

This is the workflow you'll run dozens of times a week. We'll walk through it slowly the first time.

5.1 Start at Investigations

Click **Investigations** in the sidebar (or press ⌘2).

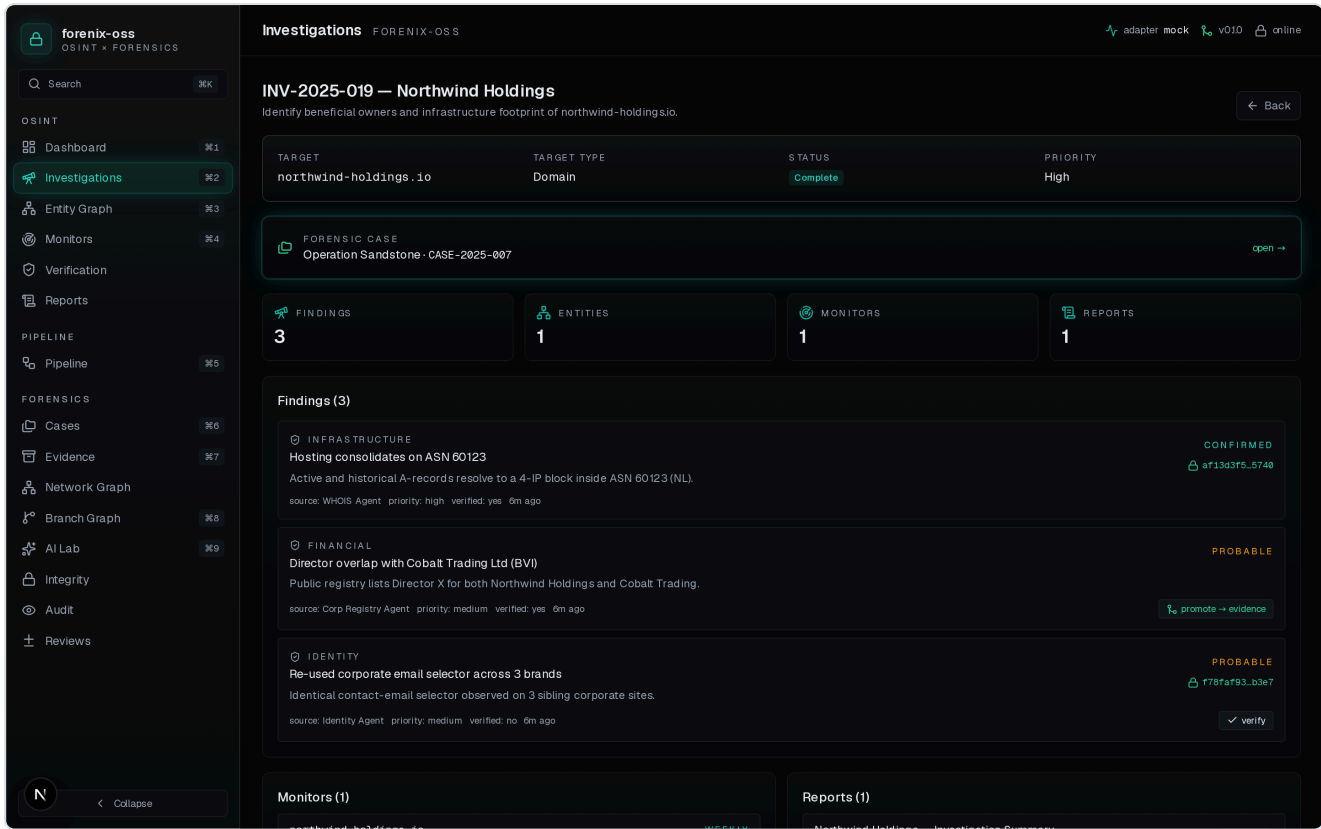


What you see:

- A table of every investigation, newest first.
- Each row has its title, target, status, finding count, a bridge chip (if it's been linked to a forensic case), and the time it was last updated.
- A filter input above the table lets you type to narrow the list — it matches against title, target, target type, status, and priority.
- A **+ New** button opens the create-form.

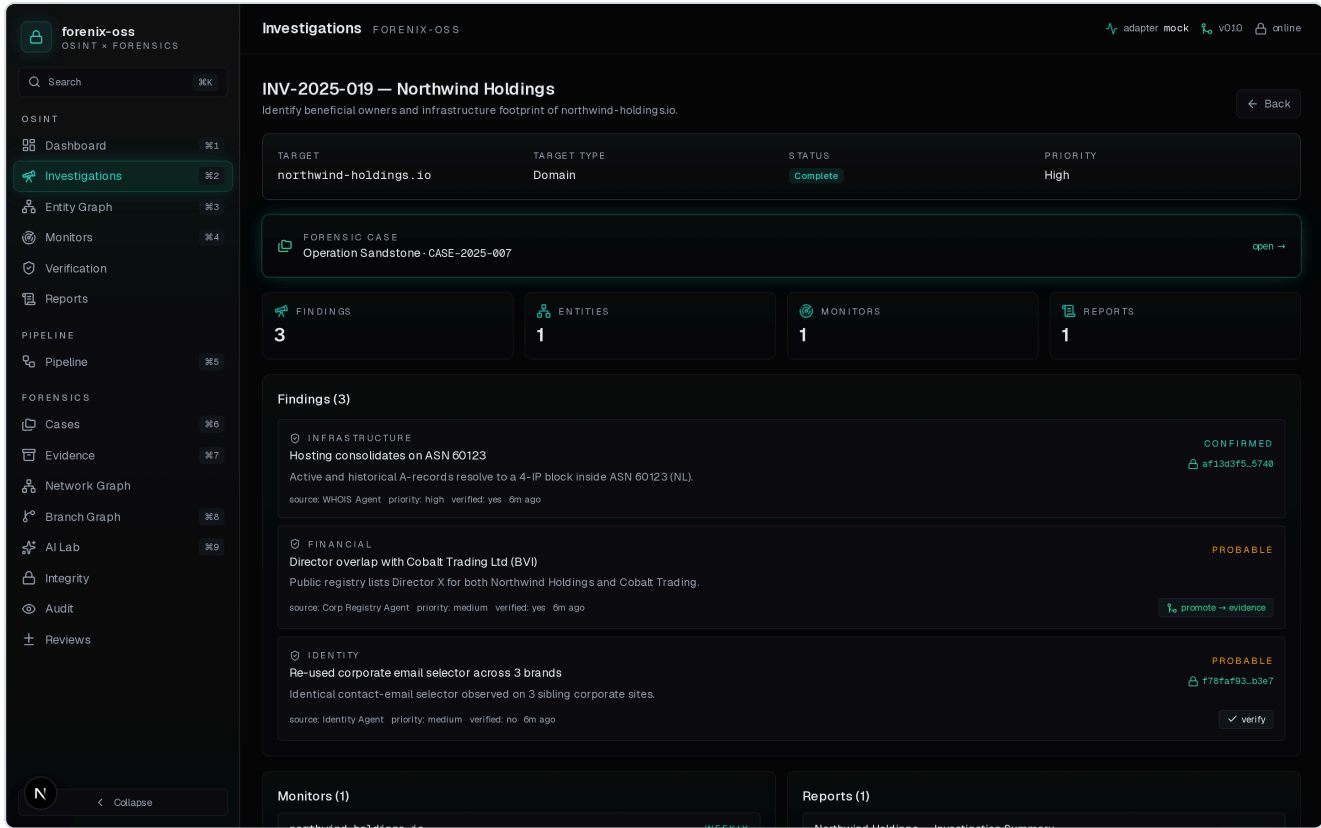
5.2 Open one to see the detail

Click any row — let's pick **INV-2025-019 — Northwind Holdings**.



What you're looking at:

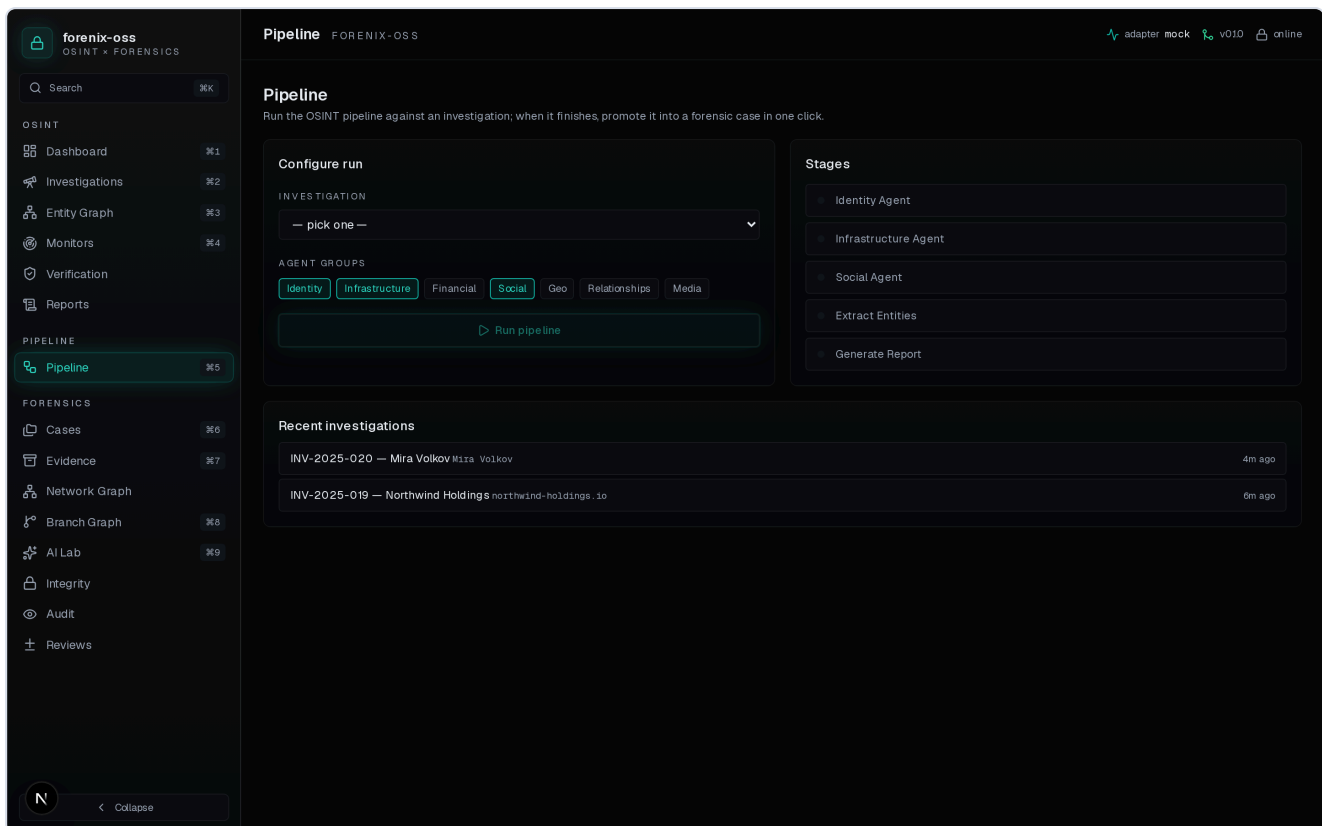
- **Header strip** — target, target type, status, priority. All the at-a-glance metadata.
- **Bridge card (the green one)**. This investigation is already linked to `CASE-2025-007 — Operation Sandstone` . Click it and the view switches straight into the case.



- **Stat row** — findings, entities, monitors, reports for this investigation.
- **Findings list** — every signal an agent produced, grouped by category. Each finding shows its confidence chip, priority, source, the reasoning trace, and (if it's been promoted) the evidence hash.
- **Action buttons on each finding row:**
 - **verify** — flips the finding's confidence from `unverified` to `confirmed` and records you as the verifier. Writes a `verify_finding` audit row.
 - **promote** → **evidence** — only visible if the investigation is bridged to a case AND the finding hasn't been promoted yet. Creates an Evidence row with its own initial commit and stamps `Finding.evidenceId`.
- **Monitors / Reports** — two side-by-side cards listing the monitor cadences and the published reports.
- **Audit trail** (bottom) — every state change for this investigation, in chain order.

5.3 Run the pipeline against a fresh target

Switch to the **Pipeline** view (sidebar, or ⌘5).

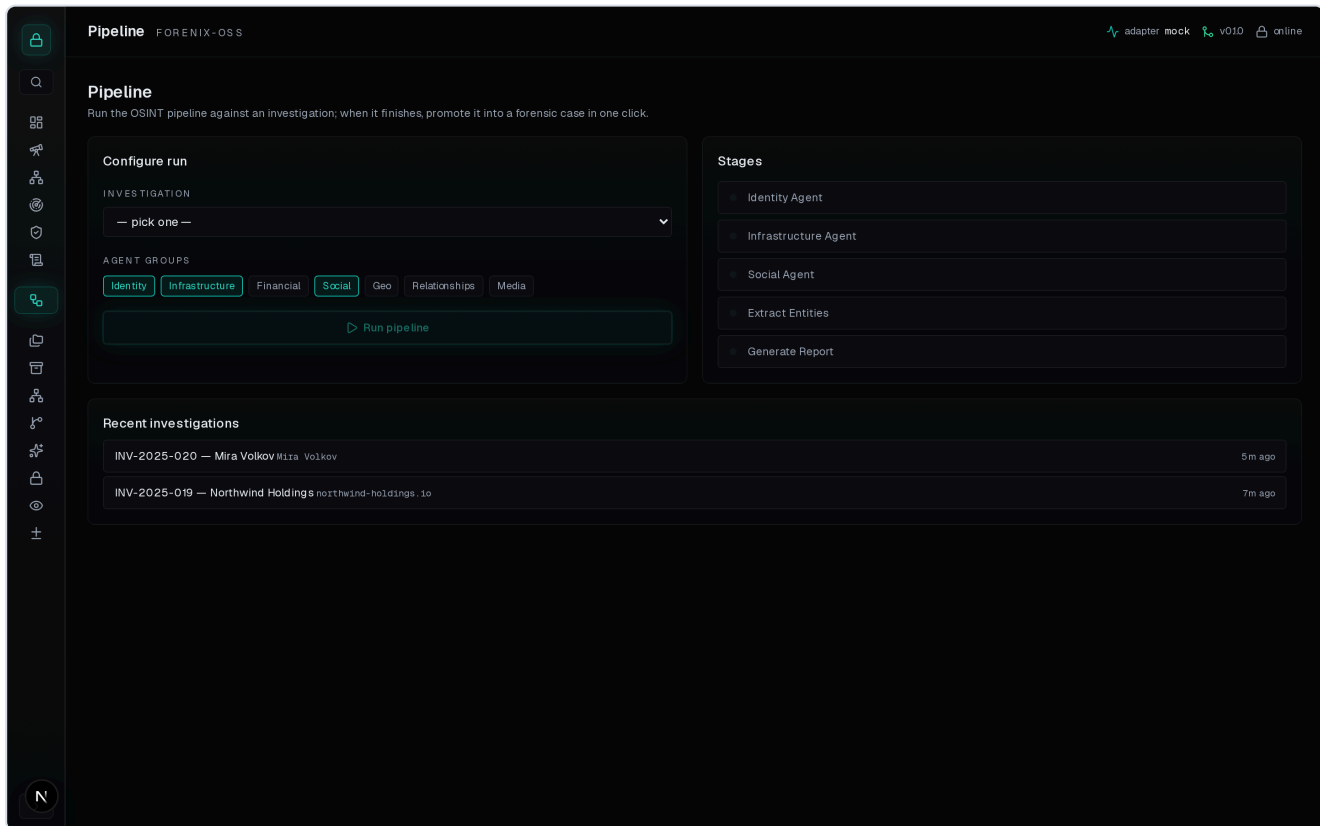


The Pipeline runner is the headline feature. It takes an existing investigation, fans out the AI agents in parallel against the target, consolidates the findings, extracts entities, and generates a draft report — all auditable, all in one button.

To run:

1. Pick an investigation from the dropdown (or click one in *Recent investigations* below — clicking sets the selection).
2. Toggle the agent groups you want to run. The defaults are `identity`, `infrastructure`, `social` — three covers most targets.

3. (Optional) override the AI adapter by passing `?adapter=nvidia` in the URL or `adapter:"nvidia"` in the body if you're calling the API directly. By default the active adapter is the one in `.env`.
4. Click **Run pipeline**.



While the pipeline runs:

- Each stage in the right-hand panel transitions `idle → running → done`.
- A `Loader2` spinner shows what's currently in flight.

When the run completes (typically 30–90 seconds on hosted LLMs), the *Pipeline complete* card appears with the counts:

- **findings** — how many discrete signals were produced.
- **entities** — how many people / orgs / domains / IPs were extracted from those findings.
- **relations** — how many directed connections between entities were inferred.
- **report** — a draft markdown report tied to the investigation.

The next step — and this is what makes forenix-oss different — is the **Open forensic case** → button.

6. Workflow 2 — Bridge findings to a forensic case

This is the workflow that turns OSINT into evidence.

6.1 What the bridge does

Clicking **Open forensic case** → does five things in one transaction:

1. **Mints a Case** with an auto-generated `CASE-YYYY-NNN` number, status `open`, priority inherited from the investigation.
2. **Creates a `main` branch** on that case (forensic cases are structured like a Git repo).
3. **Sets `Investigation.caseId`** — the link that powers the green bridge chip you saw earlier.
4. **Promotes every finding** to an Evidence row, computes a SHA-256 over the finding's content, and adds an initial `add:` commit to the case's `main` branch.
5. **Stamps `Finding.evidenceId`** so each finding now points at the evidence row it became.

Every one of those five steps writes an audit row. The chain stays valid throughout.

6.2 The bridged case detail

After the bridge, you land on the new case. It looks like this:

The screenshot displays the 'forenix-oss' web application. The sidebar on the left contains navigation links for OSINT, Pipeline, and Forensics. The main content area is titled 'Cases' and shows details for 'INV-2025-020 — Mira Volkov — forensic follow-up'. The case header includes a table with columns for Case #, Status (Open), Priority (Medium), and Progress (5%). Below this is a 'Linked investigations (1)' section showing a link to 'INV-2025-020 — Mira Volkov'. A statistics row shows 14 Evidence items, 1 Branch, 0 Merge Requests, and 0 Assignees. The 'Evidence (14)' section lists two evidence items, each with a commit chain showing the history of changes.

Top to bottom:

- **Case header** — number, status, priority, progress bar.
- **Linked investigations card** — every investigation that feeds this case. Click any row to jump back to its investigation detail.
- **Stat row** — evidence, branches, merge requests, assignees.
- **Evidence list** — and this is the heart of the page. Each evidence row shows:
 - name, type, MIME, description,
 - **truncated SHA-256 hash** in monospace,
 - **status chip** — `collected` / `verified` / `sealed`,
 - a **Seal** button (we'll get to it),

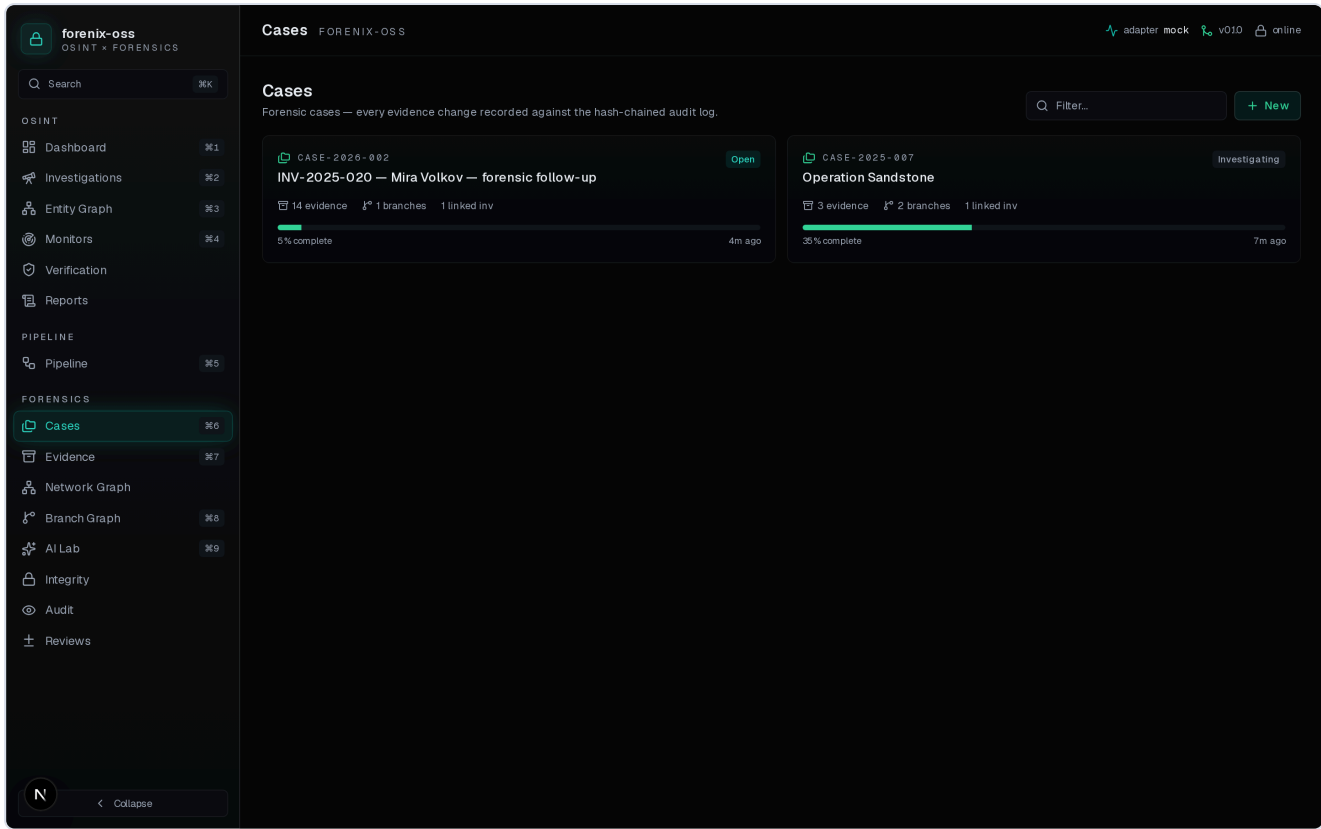
- and the **full commit chain inline**.

The screenshot displays the 'forenix-oss' web application interface. On the left is a dark sidebar with a search bar and a list of navigation items categorized under 'OSINT' and 'FORENSICS'. The 'Cases' item is highlighted. The main content area shows the details of a specific case: 'INV-2025-020 — Mira Volkov — forensic follow-up'. Below the case title, it states 'Forensic case opened from investigation INV-2025-020 — Mira Volkov Target: Mira Volkov'. A table shows case metadata: CASE # (CASE-2026-002), STATUS (Open), PRIORITY (Medium), and PROGRESS (5%). Below this, a 'Linked investigations (1)' section shows a link to 'PERSON' with details 'INV-2025-020 — Mira Volkov' and a 'Complete' status. A summary row displays counts: EVIDENCE (14), BRANCHES (1), MERGE REQUESTS (0), and ASSIGNEES (0). The 'Evidence (14)' section lists two findings, each with a 'Sealed' status and a commit chain. The first finding is 'Finding cmp7ld6w — Mira Volkov Geographic Relevance' with a commit chain showing two entries: '521625b4..1877 ADD: promoted from finding cmp7ld6w' and '1611c8aa..9caf SEAL: seal: evidence locked, no further mutations'. The second finding is 'Finding cmp7ld6v — Mira Volkov Archive Mention' with a similar commit chain.

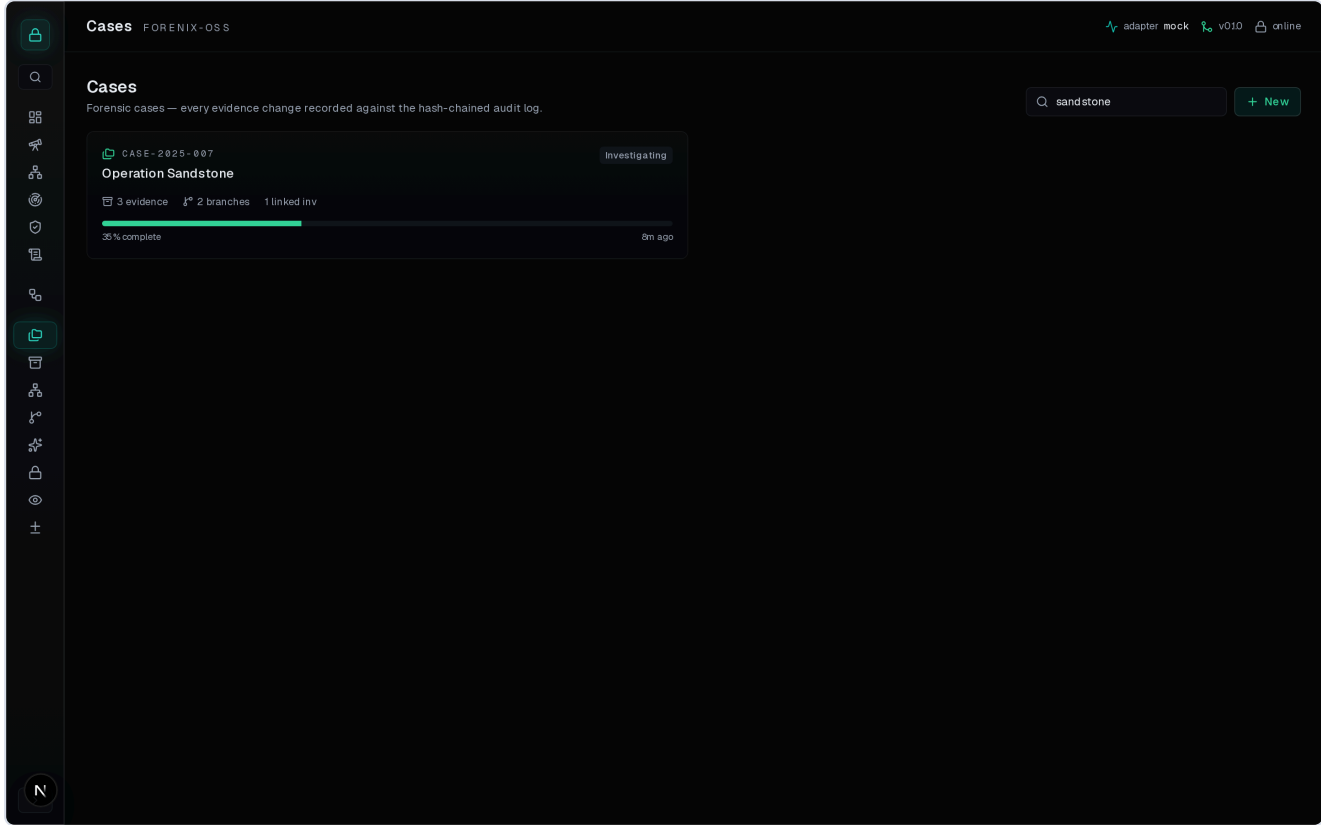
- **Branches sidebar** — every branch on this case (`main` , `evidence-review` , etc.), each colour-coded.
- **Assignees + agents** — humans and AI agents working on this case.
- **Recent audit entries** — the last 10 audit-chain rows scoped to this case.

6.3 The Cases list

If you'd rather start from the Cases view, click **Cases** (§6).



Cards show case number, title, status, progress bar, and high-level counts. Click any card to drill in. The inline filter matches across `title`, `caseNumber`, `status`, and `priority`.



7. Workflow 3 — Evidence + chain of custody

Evidence is the unit of forensic work. Each Evidence row carries a content hash + a commit history. Forenix-oss is one of the very few platforms where every change to a piece of evidence shows up on a tamper-evident chain.

7.1 The Evidence inventory

Switch to **Evidence** (⌘7).

forenix-oss
OSINT × FORENSICS

Search

⌘K

OSINT

Dashboard

⌘1

Investigations

⌘2

Entity Graph

⌘3

Monitors

⌘4

Verification

Reports

PIPELINE

Pipeline

⌘5

FORENSICS

Cases

⌘6

Evidence

⌘7

Network Graph

Branch Graph

⌘8

AI Lab

⌘9

Integrity

Audit

Reviews

N

< Collapse

EvidenceFORENIX-OSS

adapter mock

v010

online

Evidence

Every piece of evidence across every open case. Each row links back to its parent case and its hash-chained commits.

Filter...

17 items

NAME	TYPE	SIZE	HASH
Finding cmp7ld6w — Mira Volkov Geographic Relevance Mira Volkov may be relevant to geographic analysis or operations	document application/x-osint-finding	—	6c0766f1c804 SHA-256
Finding cmp7ld6v — Mira Volkov Archive Mention Mira Volkov is mentioned in an archive snapshot related to geo context	document application/x-osint-finding	—	807f9257165 SHA-256
Finding cmp7ld6v — Mira Volkov Geo Location Mira Volkov is associated with a geographic location	document application/x-osint-finding	—	872834d9e00 SHA-256
Finding cmp7ld6v — Mira Volkov Information Availability Information about Mira Volkov is available through online searches and archives, indicating a certain level of public visibility	document application/x-osint-finding	—	a1526743684 SHA-256
Finding cmp7ld6v — Mira Volkov Potential Network Mira Volkov may be connected to other individuals or groups through her social media presence, as suggested by the search results and archive snapshot	document application/x-osint-finding	—	6594588f8f5a SHA-256
Finding cmp7ld6v — Mira Volkov Archive Snapshot An archive snapshot of Mira Volkov's social context is available at https://archiveexamplelocal/social/Mira%20Volkov providing a historical record of her online activity	document application/x-osint-finding	—	e2ae4caf0007 SHA-256
Finding cmp7ld6v — Mira Volkov Social Media Presence Mira Volkov has an online presence on social media platforms, as indicated by the search results on https://searchexamplelocal/q=Mira%20Volkov&g=social	document application/x-osint-finding	—	f9a3cf22eecd SHA-256
Finding cmp7ld6u — Identity Context Relevance Mira Volkov is mentioned in the context of identity, which may imply relevance to personal, social, or professional identification	document application/x-osint-finding	—	10ad91a43020 SHA-256
Finding cmp7ld6u — Potential Online Presence Given the search result and archive mention, Mira Volkov may have an online presence or history of activity	document application/x-osint-finding	—	7dfc7c9d9771 SHA-256
Finding cmp7ld6u — Mira Volkov Archive Mention An archive snapshot exists that mentions Mira Volkov in an identity context, suggesting historical or recorded relevance	document application/x-osint-finding	—	7d78f66cd770 SHA-256
Finding cmp7ld6u — Mira Volkov Identity Signal A search result for Mira Volkov is present on the example local search engine, indicating potential online presence or activity	document application/x-osint-finding	—	bb0476d7620f SHA-256

The Evidence view shows every piece of evidence across every case. Useful when you want to find something by hash, or inventory the state of an entire investigation.

Columns: name, type/MIME, size, **hash** (truncated), status, parent case, commit count, added-at. Filter works across all of these.

7.2 Seal an evidence row

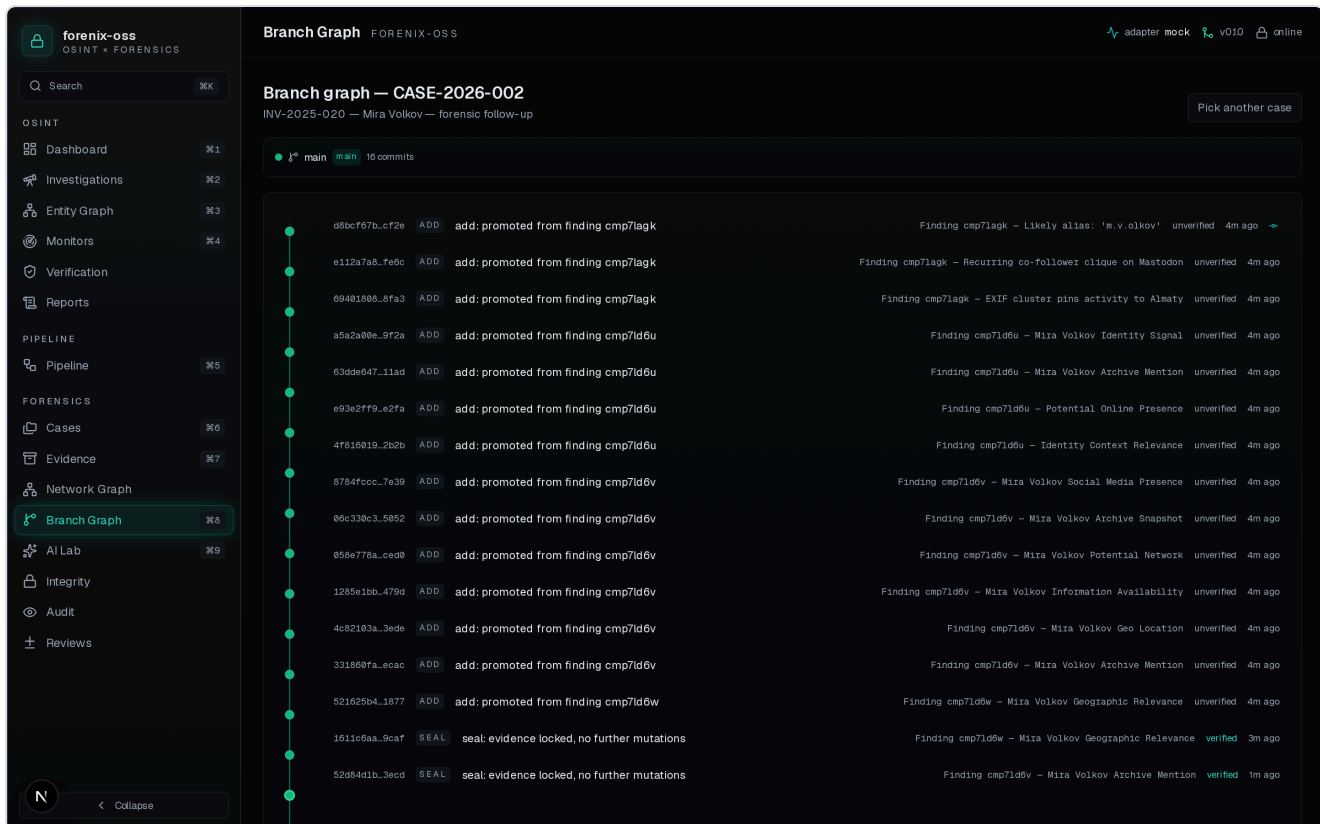
A **sealed** evidence row is immutable — no further commits are accepted, and the row's hash is locked into the chain.

To seal, click the **Seal** button on the right side of any evidence card on the case-detail page. The status chip flips to **sealed**, a **seal** commit is written to **main**, and an audit row is added.

This is the action you take when the case team has signed off and the evidence is "ready for court."

7.3 The Git-style branch graph

For visual reasoning about the commit history, switch to **Branch Graph** (⌘8) and pick the case you want to inspect.



What you see:

- One **swimlane per branch**, leftmost is `main`.
- One **dot per commit**, coloured to match the branch.
- Verified commits get a **green ring** around the dot.
- The right-hand panel lists each commit's hash, change type (`add` / `verify` / `seal` / `merge`), evidence it touched, verified state, and time.

This is the page you'd put in front of a sceptical reviewer.

8. Workflow 4 — Verify the audit chain

The audit chain is what makes forenix-oss claim "court-grade custody." It's a SHA-256 forward chain over every state-changing write.

8.1 Open the Audit view

Click **Audit** in the sidebar.

Audit log
Append-only chain across both workflows. Every write computes sha256(prevHash | action | entity | entityid | iso(t)).

Search: action, entity, hash... 21 entries

HASH	ACTION	ENTITY	SCOPE	WHEN
→ b84a5d97...58b8 ← 00000000...0000	create_user	User cmp71aghi000	—	8m ago
→ b405962f...13b7 ← b84a5d97...58b8	create_user	User cmp71aghi000	—	8m ago
→ 02409c9c...2e6e ← b405962f...13b7	create_user	User cmp71aghi000	—	7m ago
→ 0bae3843...57a2 ← 02409c9c...2e6e	create_investigation	Investigation cmp71agk5001	Inv	7m ago
→ 1e80e7a4...3fa0 ← 0bae3843...57a2	run_pipeline	Investigation cmp71agk5001	Inv	7m ago
→ c0893b89...0860 ← 1e80e7a4...3fa0	create_case	Case cmp71agi3000	case	7m ago
→ 72715b02...1df8 ← c0893b89...0860	promote_to_evidence	Finding f-1	case	7m ago
→ 0996a640...2dcc ← 72715b02...1df8	verify_evidence	EvidenceCommit cmp71agk000	case	7m ago
→ 3de0414a...cace ← 0996a640...2dcc	generate_report	Report r-1	Inv	7m ago
→ 023e80f8...c7db ← 3de0414a...cace	pipeline_started	Investigation cmp71agk001	Inv	5m ago
→ 3dfc7f00...a1be ← 023e80f8...c7db	agent_identity_completed	Investigation cmp71agk001	Inv	5m ago
→ a5034c70...abe7 ← 3dfc7f00...a1be	agent_social_completed	Investigation cmp71agk001	Inv	5m ago
→ 9ef2a8b2...a64f ← a5034c70...abe7	agent_geo_completed	Investigation cmp71agk001	Inv	5m ago
→ 750f7d60...18ed ← 9ef2a8b2...a64f	entities_extracted	Investigation cmp71agk001	Inv	4m ago
→ 8aaa0423...f730 ← 750f7d60...18ed	report_generated	Report r-1	Inv	4m ago

The table shows every audit row in chain order. Per row:

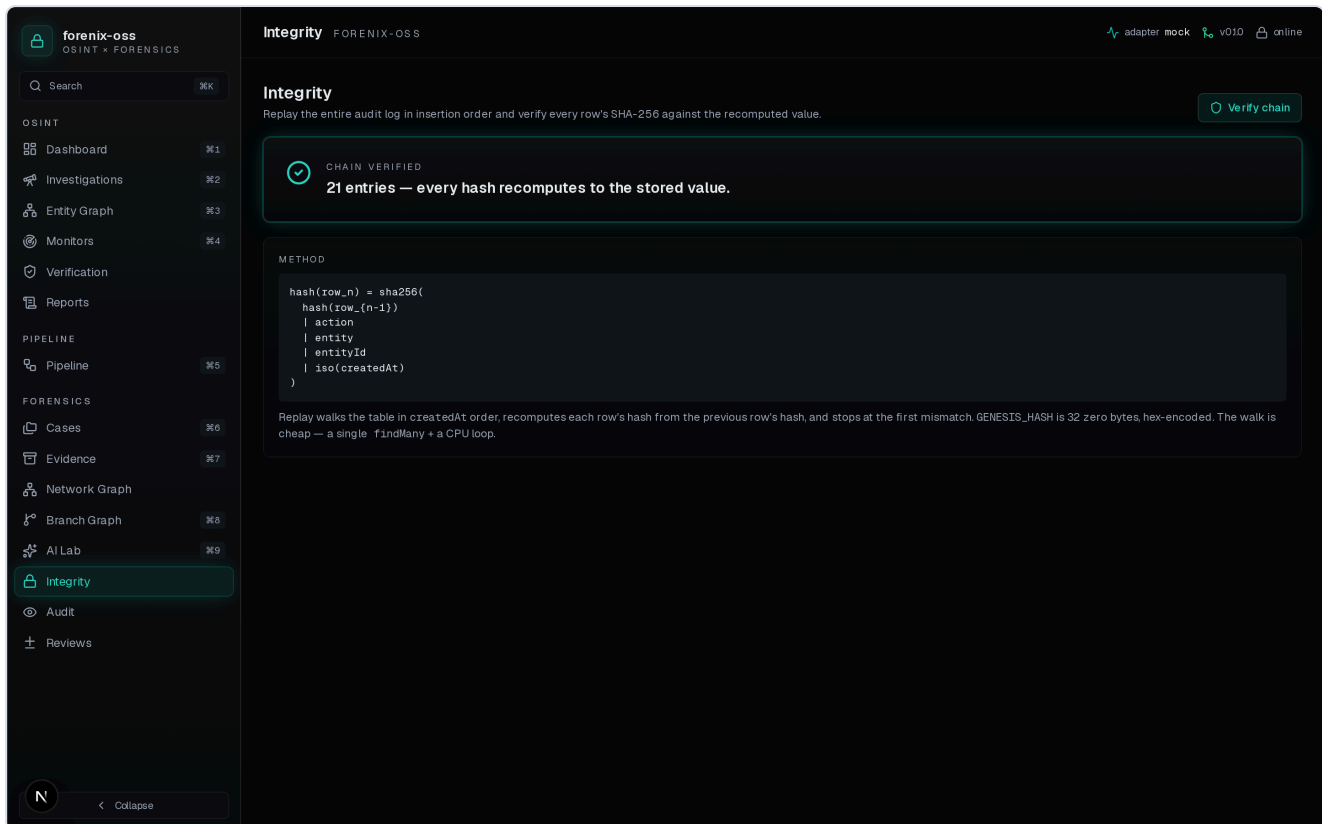
- the SHA-256 hash of the row itself (monospace, truncated for display);
- the **previous row's hash** inline directly below — visual proof of the chain;
- the action (`pipeline_started` , `agent_identity_completed` , `promote_finding_to_evidence` , `seal_evidence` , ...);
- the entity it acted on (Investigation / Case / Finding / ...);
- the scope — clickable jump links to the investigation and/or case;
- the relative time.

The filter input matches on action, entity, hash. Type `seal` to see every sealing event; type `pipeline` to see every pipeline event; etc.

A row whose `prevHash` doesn't equal the previous row's `hash` gets a red tint automatically.

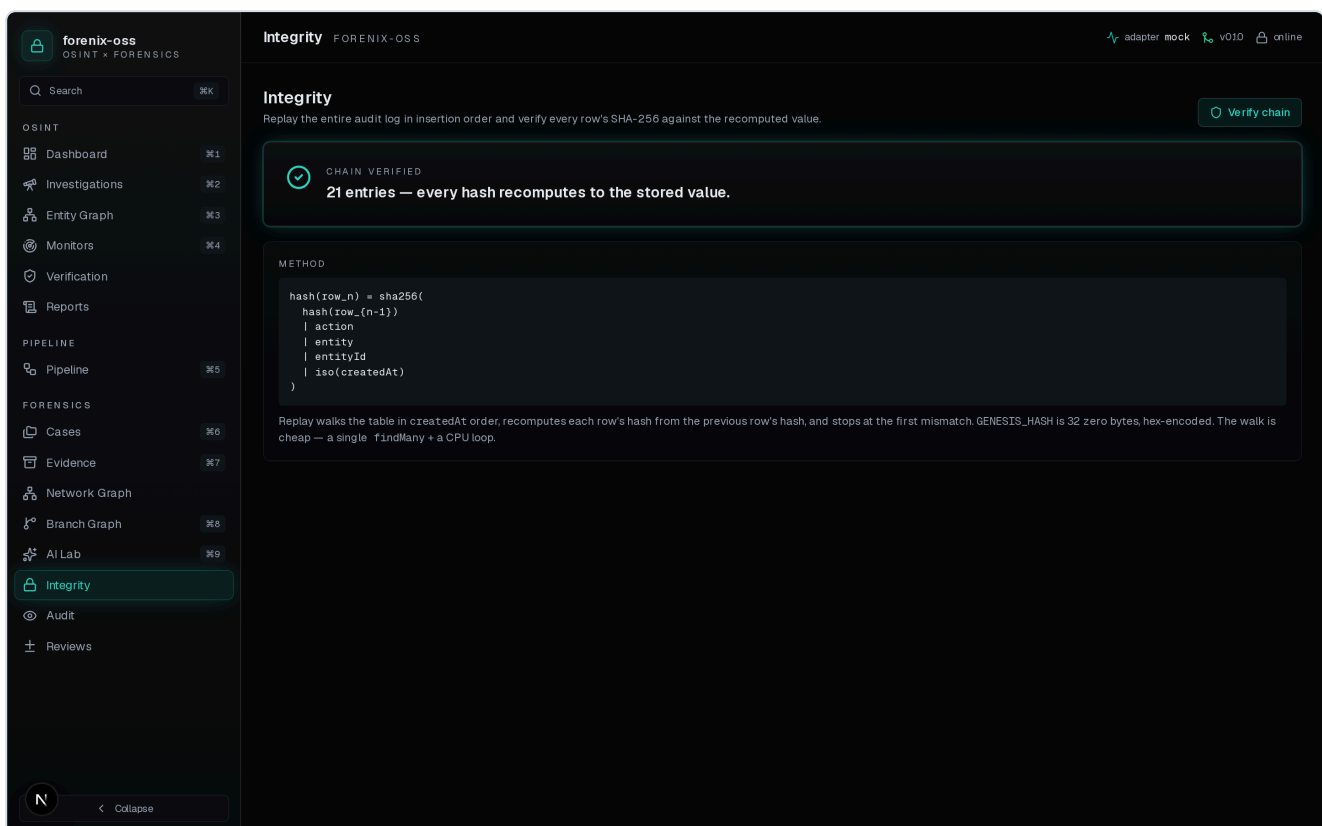
8.2 Verify the chain end-to-end

Now click **Integrity**.



The page shows the method (the SHA-256 input ordering), but no verdict yet. Click **Verify chain**.

The button calls `GET /api/audit/verify` which walks the entire table in `createdAt` order, recomputes every row's hash from the previous row's `hash`, and reports the result.



The green panel above is the real screenshot from a verified chain on the demo dataset — *21 entries, every hash recomputes to the stored value.*

If verification fails, the panel turns red and shows the row id where the chain broke, the expected hash, and the stored hash — all the information a forensic investigator needs to triage the event.

8.3 Verifying the chain offline

Anyone with read access to the database can verify the chain without our cooperation. From `docs/07-SECURITY.md` :

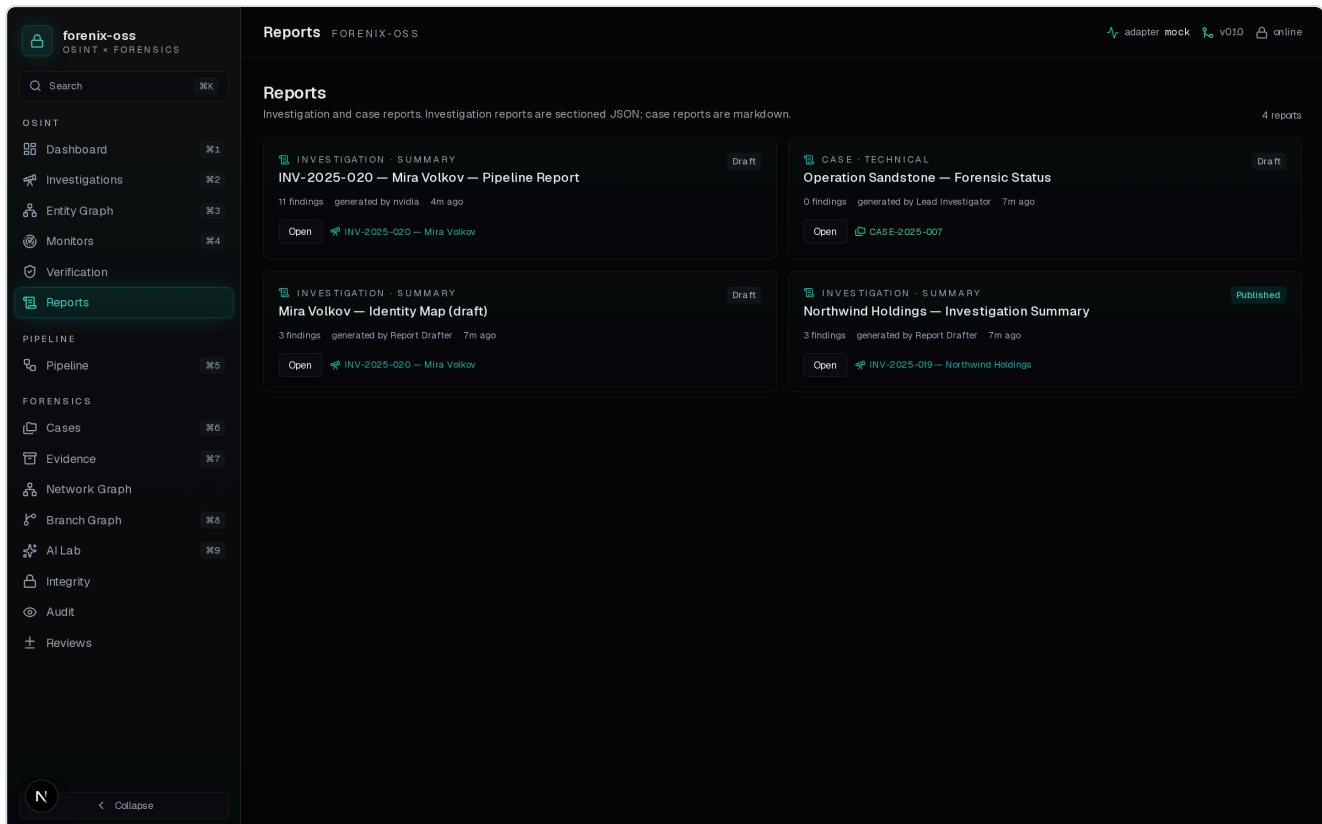
```
import csv, hashlib
GENESIS = "0" * 64
prev = GENESIS
for r in csv.DictReader(open("audit.csv")):
    h = hashlib.sha256("|".join([
        prev, r["action"], r["entity"], r["entityId"] or "",
        r["createdAt"]
    ]).encode("utf-8")).hexdigest()
    assert r["prevHash"] == prev and r["hash"] == h, f"BROKEN at {r['id']}"
    prev = r["hash"]
print("chain OK")
```

12 lines of Python. That's the entire trust anchor.

9. Workflow 5 — Generate and read reports

9.1 The Reports list

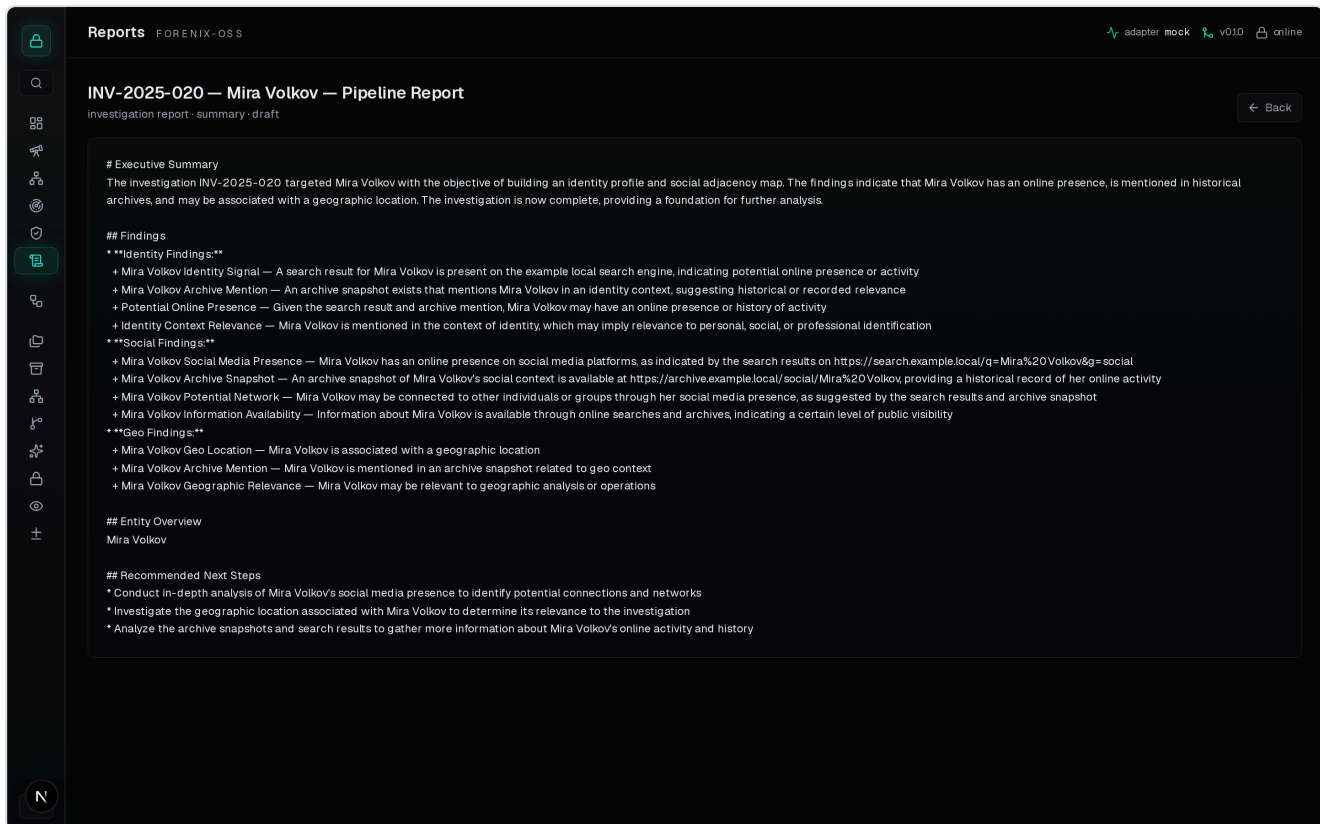
Click **Reports** in the sidebar.



The list shows every report — both *investigation* reports (sectioned JSON, the Argus shape) and *case* reports (markdown, the ForenX shape). Each card shows source, status, finding count, generator, and timestamps. Click **Open** to read.

9.2 Reading a report

The detail view auto-detects whether the report has `sections` (JSON) or `content` (markdown) — or both — and renders each accordingly.



For an analyst, this is the closest thing to a "deliverable" — copy/paste the section text into your write-up, or export the markdown directly.

9.3 How a report gets generated

Reports are produced automatically as the *last stage* of the Pipeline runner. The active AI adapter is called with `generateReport(investigation, findings)` and asked to produce a markdown report in this shape:

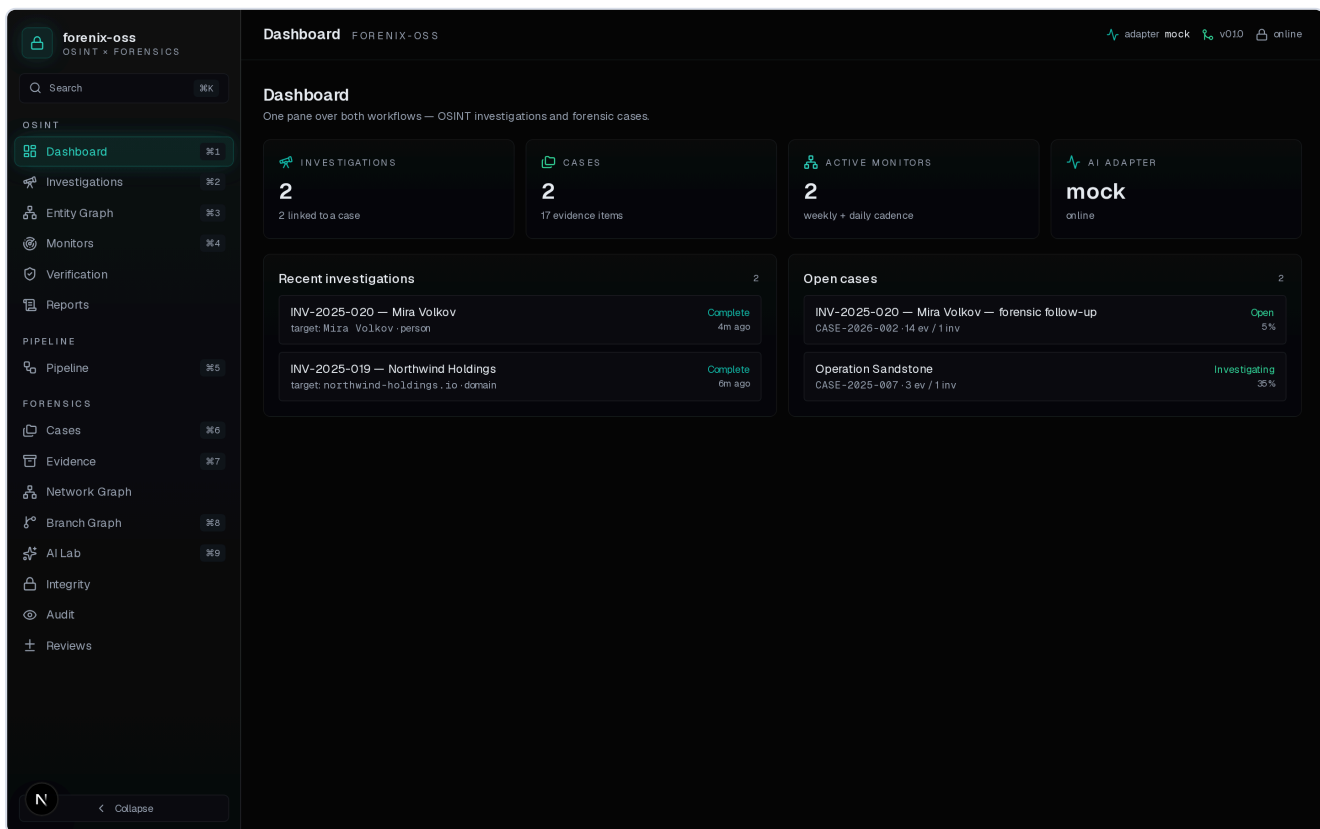
- **# Executive Summary** — 3-5 sentences.
- **## Findings** — grouped by agent group.
- **## Entity overview** — names mentioned in the findings.
- **## Recommended next steps** — 3-5 bullet points.

The report is stored on the `Report` table with `source = "investigation"` and gets attached to the parent investigation. You don't need to do anything — just run the pipeline.

10. Feature reference — every view

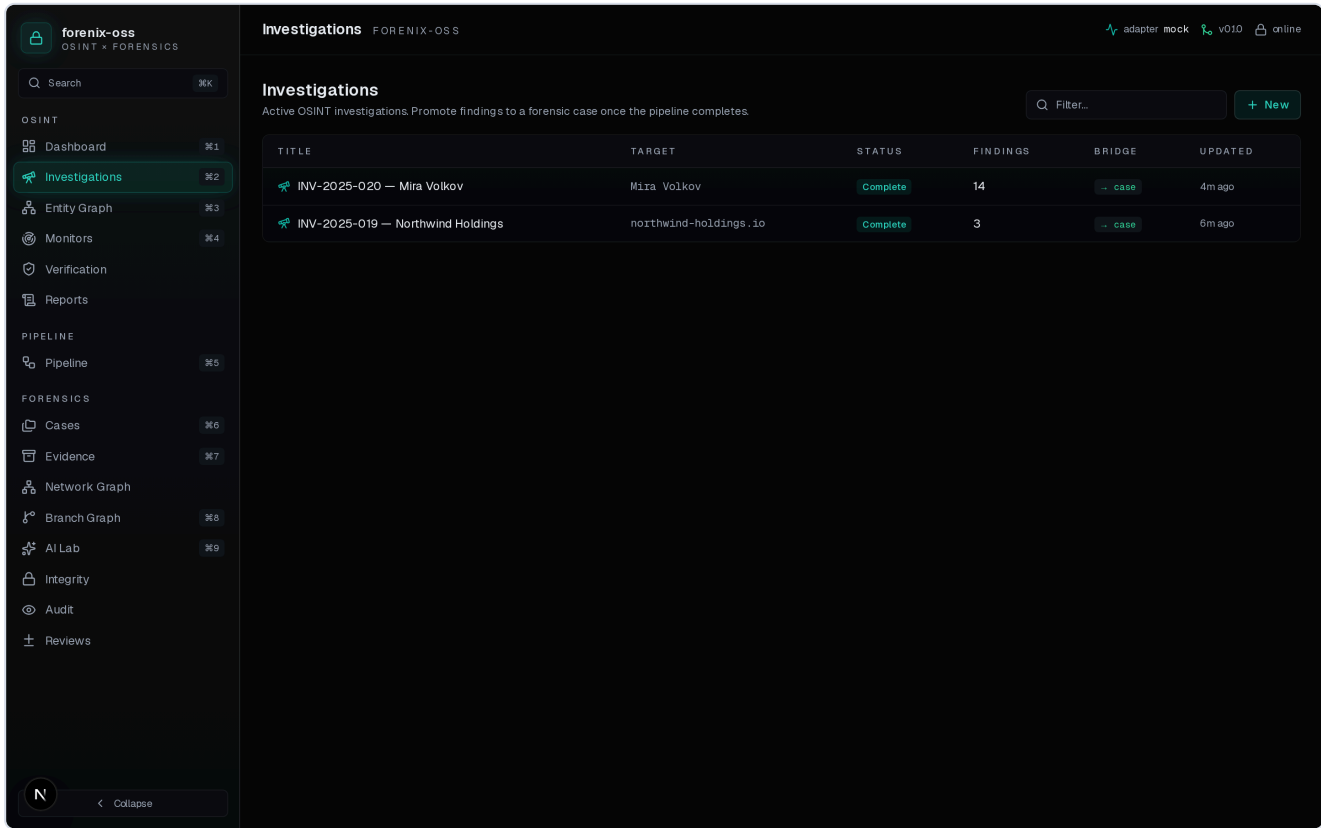
This section is a sweep across every view. For each one we explain *what it is*, *what problem it solves*, and *what it deliberately does not claim to do*.

10.1 Dashboard



- **What:** four stat cards + a *Recent investigations* preview and an *Open cases* preview.
- **Solves:** the analyst's "where do I start today?" problem.
- **Does not:** show charts/trendlines. No per-user scope (you see everything you have access to).

10.2 Investigations

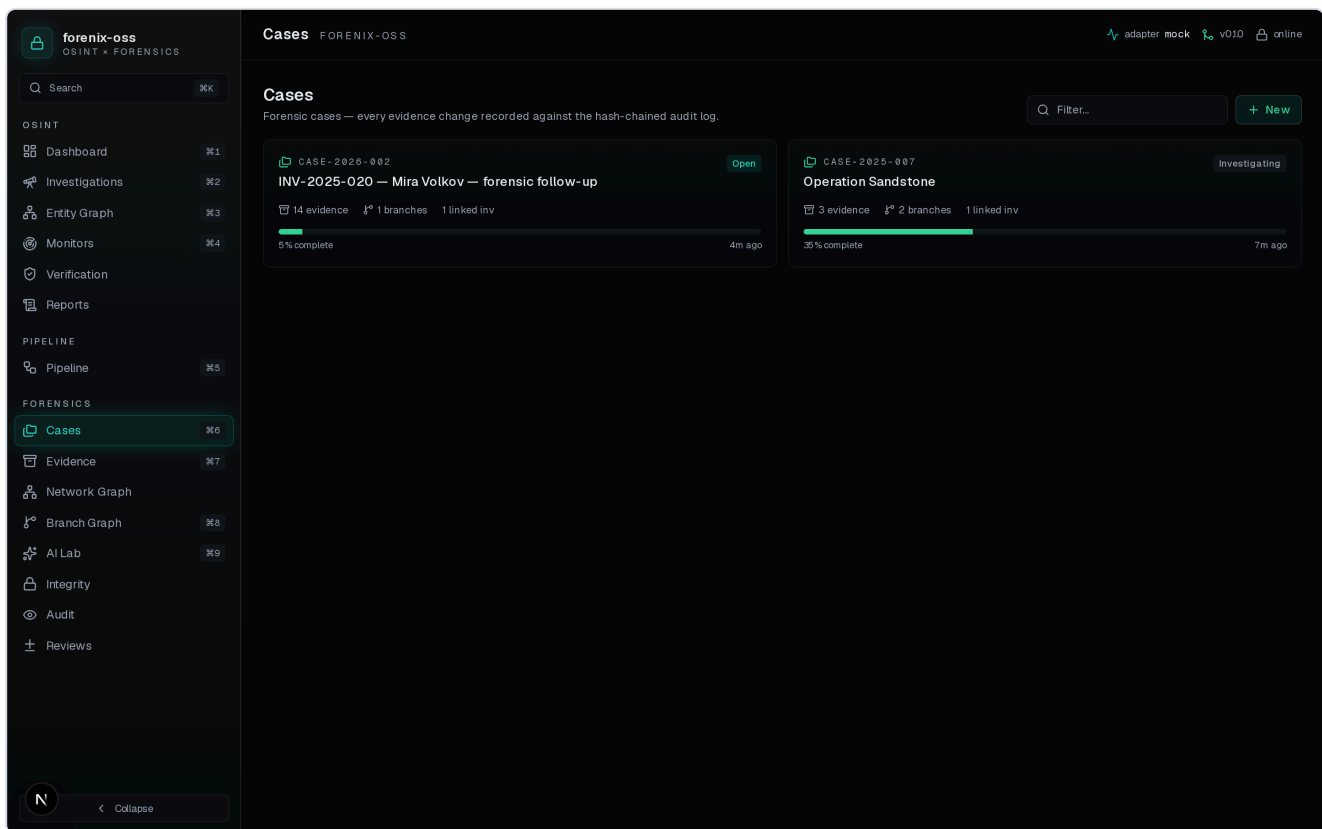


- **What:** the OSINT collection workspace.
- **Solves:** the iterative back-and-forth of building a profile.
- **Does not:** support bulk-edit on findings; per-agent re-runs aren't exposed (you can re-run the whole pipeline).

10.3 Pipeline

- **What:** the runner that drives the AI adapter end-to-end.
- **Solves:** turning a target into a structured set of findings with provenance.
- **Does not:** stream progress (stages animate in the UI but the server-side run is sequential per agent group).

10.4 Cases



- **What:** the forensic case manager.
- **Solves:** keeping evidence, branches, MRs, and assignments organised under one number.
- **Does not:** support file-byte upload (Evidence carries metadata + hash; bytes will land in Phase 8).

10.5 Evidence

- **What:** the cross-case evidence inventory.
- **Solves:** "find all evidence with hash starting 0ca..." / "show me every unverified item across all cases."
- **Does not:** pivot/group yet — it's a flat table.

10.6 Branch Graph

forenix-oss
OSINT x FORENSICS

Search [3K]

OSINT

- Dashboard [1]
- Investigations [2]
- Entity Graph [3]
- Monitors [4]
- Verification
- Reports

PIPELINE

- Pipeline [5]

FORENSICS

- Cases [6]
- Evidence [7]
- Network Graph
- Branch Graph [8]**
- AI Lab [9]
- Integrity
- Audit
- Reviews

N < Collapse

Branch Graph FORENIX-OS S

adapter mock v010 online

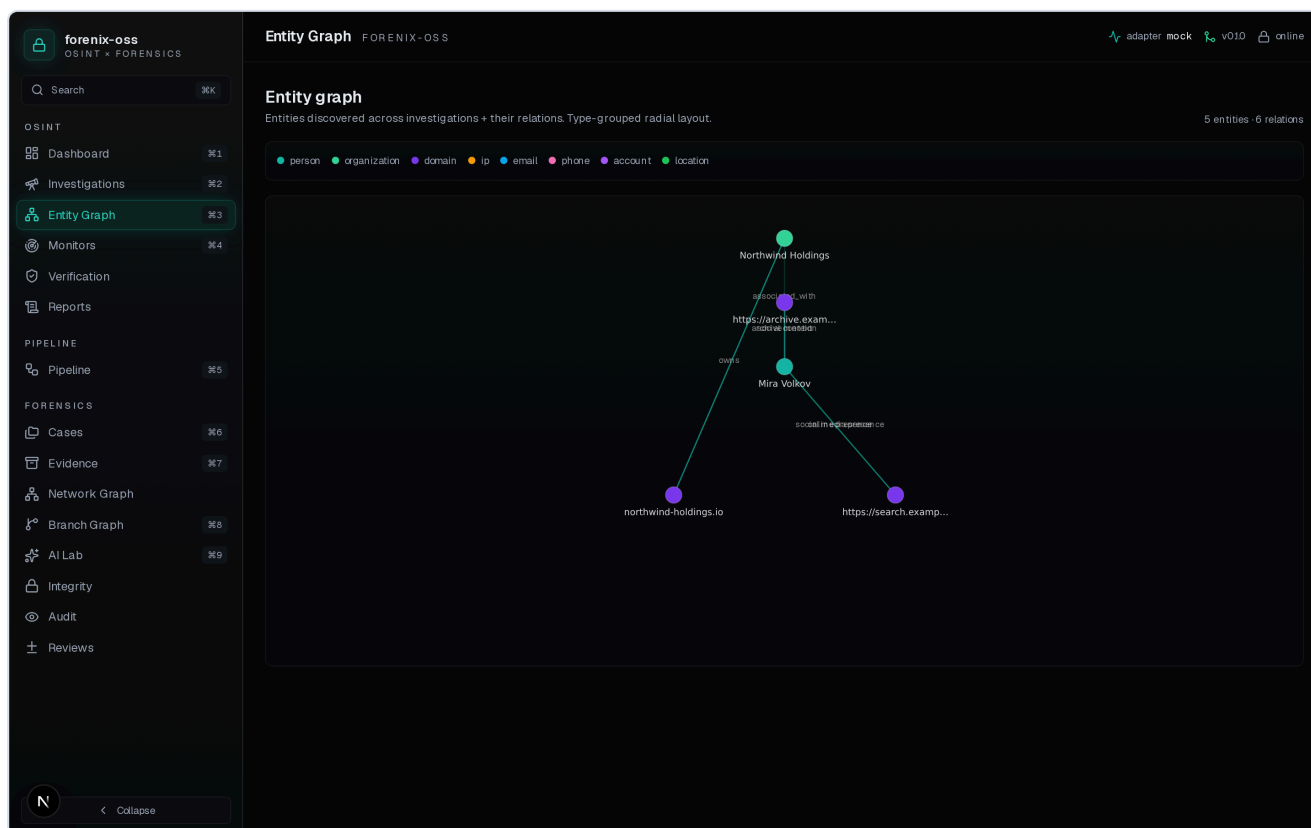
Branch graph — CASE-2026-002
INV-2025-020 — Mira Volkov — forensic follow-up [Pick another case](#)

main 16 commits

Commit Hash	Status	Description	Findings	Verification	Time
d8bcf67b..cf2e	ADD	add: promoted from finding cmp7lagk	Finding cmp7lagk - Likely alias: 'm.v.olkov'	unverified	4m ago
e112a7a8..fe6c	ADD	add: promoted from finding cmp7lagk	Finding cmp7lagk - Recurring co-follower clique on Mastodon	unverified	4m ago
69401808..8fa3	ADD	add: promoted from finding cmp7lagk	Finding cmp7lagk - EXIF cluster pins activity to Almaty	unverified	4m ago
a5a2a00e..9f2a	ADD	add: promoted from finding cmp7ld6u	Finding cmp7ld6u - Mira Volkov Identity Signal	unverified	4m ago
63dde647..11ad	ADD	add: promoted from finding cmp7ld6u	Finding cmp7ld6u - Mira Volkov Archive Mention	unverified	4m ago
e93e2ff9..e2fa	ADD	add: promoted from finding cmp7ld6u	Finding cmp7ld6u - Potential Online Presence	unverified	4m ago
4f816019..2b2b	ADD	add: promoted from finding cmp7ld6u	Finding cmp7ld6u - Identity Context Relevance	unverified	4m ago
8784fccc..7e39	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Social Media Presence	unverified	4m ago
06c330c3..5052	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Archive Snapshot	unverified	4m ago
050e778a..ced8	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Potential Network	unverified	4m ago
1285e1bb..479d	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Information Availability	unverified	4m ago
4c82103a..3ede	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Geo Location	unverified	4m ago
331860fa..ecac	ADD	add: promoted from finding cmp7ld6v	Finding cmp7ld6v - Mira Volkov Archive Mention	unverified	4m ago
521625b4..1877	ADD	add: promoted from finding cmp7ld6w	Finding cmp7ld6w - Mira Volkov Geographic Relevance	unverified	4m ago
1611c6aa..9caf	SEAL	seal: evidence locked, no further mutations	Finding cmp7ld6w - Mira Volkov Geographic Relevance	verified	3m ago
52d84d1b..3ecd	SEAL	seal: evidence locked, no further mutations	Finding cmp7ld6v - Mira Volkov Archive Mention	verified	1m ago

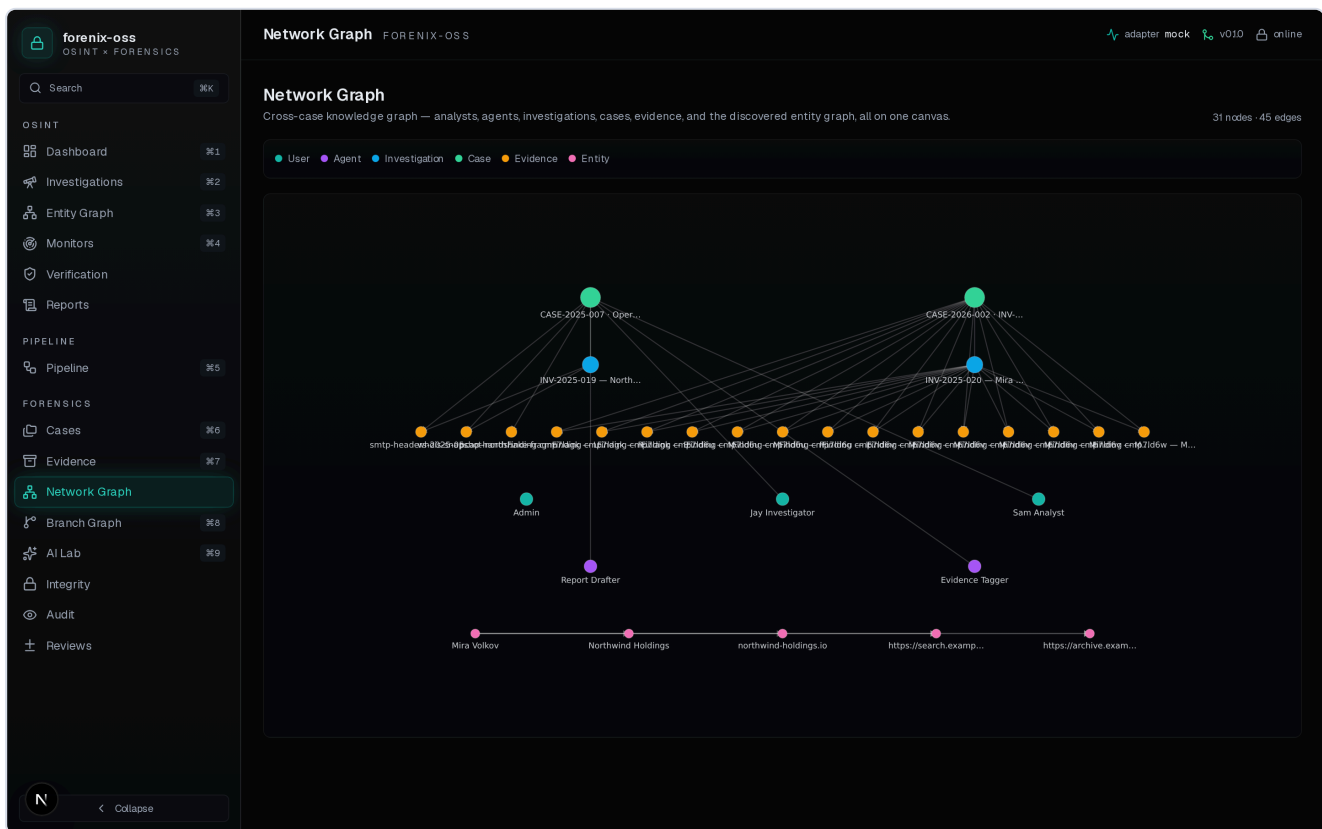
- **What:** Git-style commit graph for the active case.
- **Solves:** explaining the evidence-mutation history visually.
- **Does not:** support drag-to-reposition, no live diff view yet.

10.7 Entity Graph



- **What:** OSINT entities + their relations laid out by type.
- **Solves:** "show me everyone we've discovered around this target."
- **Does not:** force-physics; deterministic radial layout only (which is why screenshots are stable).

10.8 Network Graph



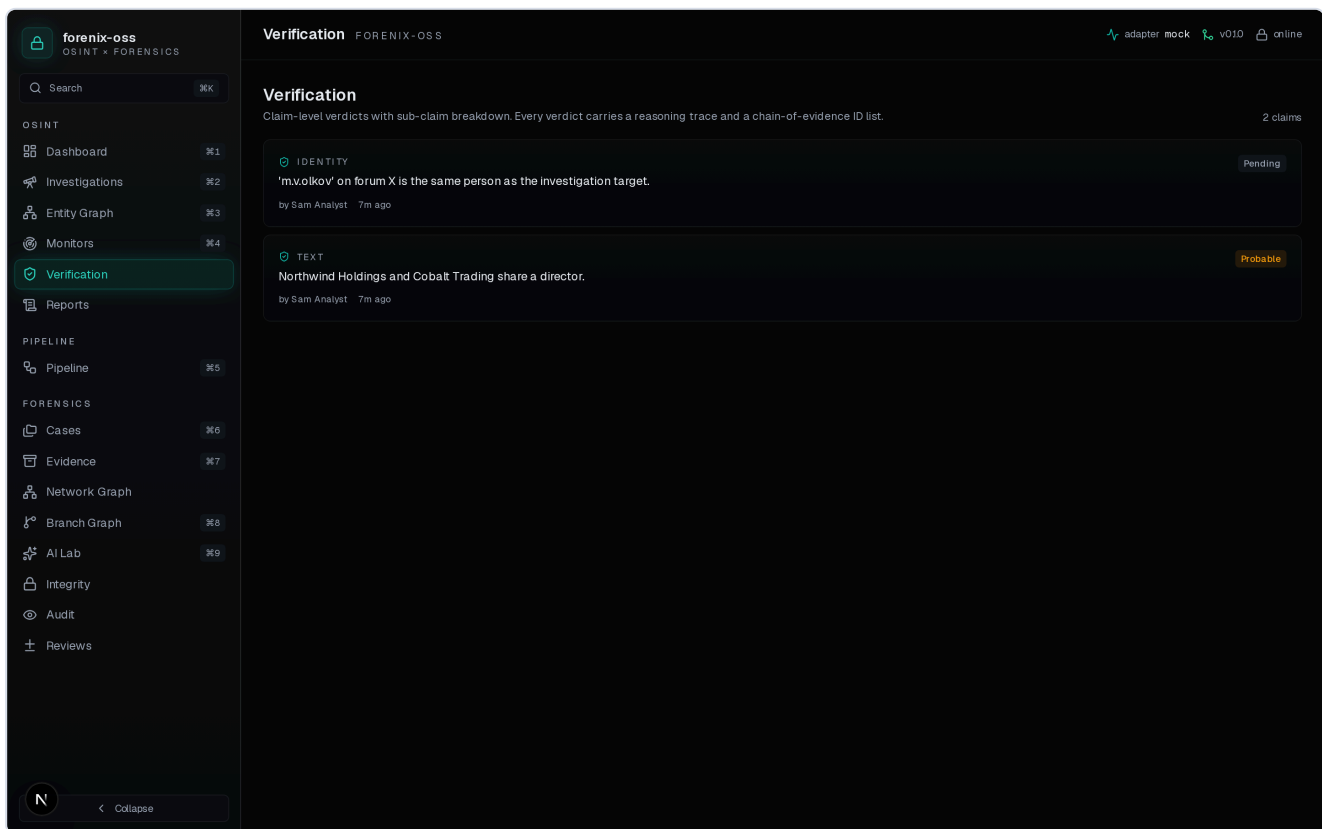
- **What:** cross-case knowledge graph — users, agents, investigations, cases, evidence, entities, all on one canvas.
- **Solves:** seeing what connects two cases or two analysts.
- **Does not:** zoom / pan yet; labels overlap above ~250 nodes.

10.9 Monitors

The screenshot shows the 'Monitors' section of the forenix-oss application. The sidebar on the left contains a search bar and a list of navigation items: OSINT (Dashboard, Investigations, Entity Graph, Monitors), Verification, Reports, PIPELINE (Pipeline), FORENSICS (Cases, Evidence, Network Graph, Branch Graph, AI Lab, Integrity, Audit, Reviews). The main content area is titled 'Monitors' and shows a list of active monitors. Two monitors are visible: 'Mira Volkov · person' (DAILY) and 'northwind-holdings.io · domain' (WEEKLY). Each monitor card displays its frequency, name, last run, next run, and recent runs. The 'Mira Volkov · person' monitor shows a last run of '—' and a next run of 'just now'. The 'northwind-holdings.io · domain' monitor shows a last run of '3d ago' and a next run of 'just now'. Below the 'northwind-holdings.io · domain' monitor, there is a section for 'RECENT RUNS' showing a single run: 'complete -1 findings' (3d ago).

- **What:** cadenced re-runs of an investigation pipeline.
- **Solves:** keeping targets fresh after the report ships.
- **Does not:** actually wake the monitor — the background scheduler is Phase 7. The data layer is ready.

10.10 Verification



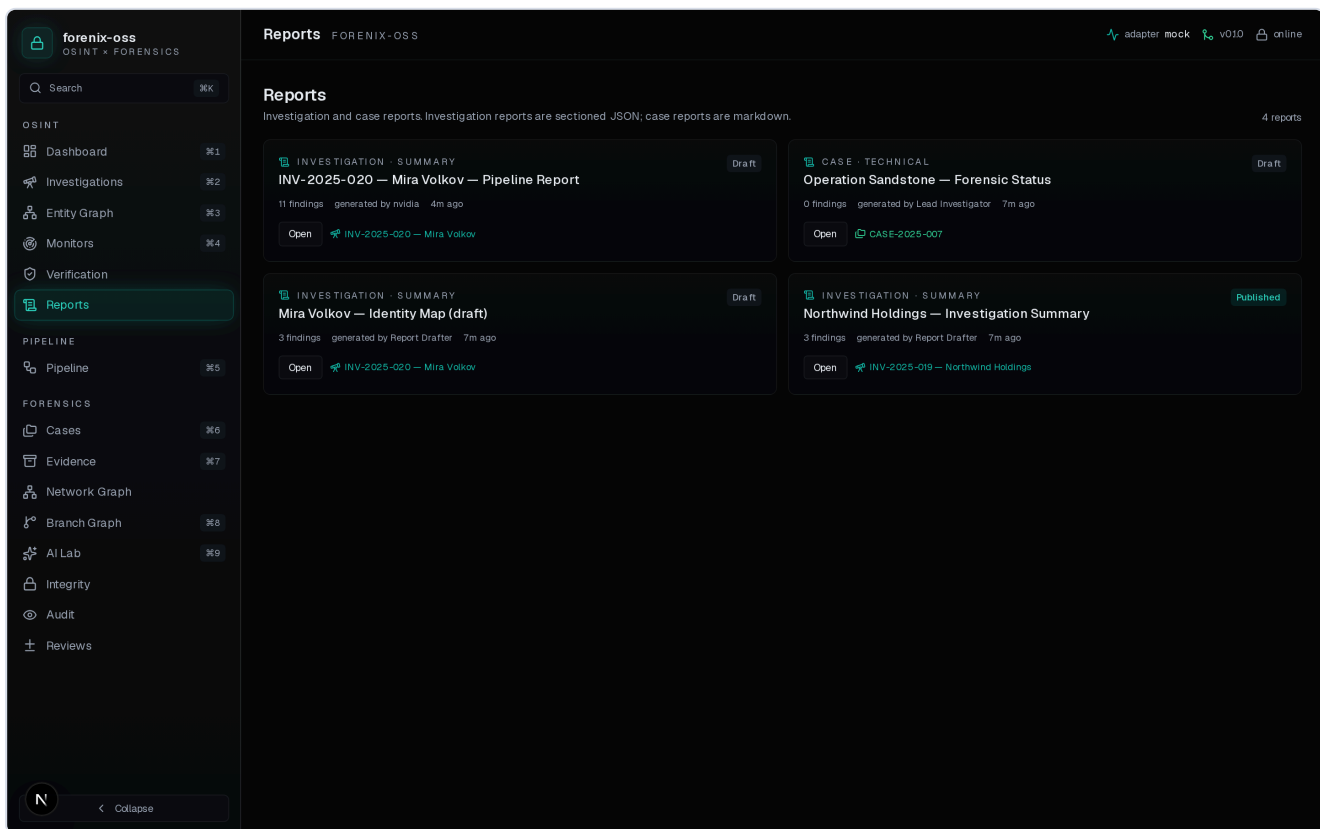
- **What:** claim-level verdicts (confirmed / probable / disputed / false).
- **Solves:** giving the human the final say over a model's confidence.
- **Does not:** support inline verdict editing yet (verdicts come from the API; UI mutations land next).

10.11 AI Lab

The screenshot displays the forenix-oss AI Lab interface. The sidebar on the left lists various tools under three categories: OSINT, PIPELINE, and FORENSICS. The AI Lab tool is selected and highlighted. The main workspace shows two active agents: 'Evidence Tagger' and 'Report Drafter'. Both agents are currently 'Idle'. Each agent has a 'RECENT TASKS' section. The Evidence Tagger's task shows a JSON output: `{'tags': ['smtp', 'relay', 'residential-proxy'], 'riskScore': 0.71}`. The Report Drafter's task shows a text output: 'Drafted a 3-section investigation summary across identity / infrastructure / financial.' The top right of the main panel indicates '2 agents' and 'online' status.

- **What:** what each agent was asked, what it returned, and how confident it was.
- **Solves:** debugging a misbehaving agent before its findings poison a case.
- **Does not:** support re-run / cancel from this page yet.

10.12 Reports



- **What:** both investigation-side and case-side reports.
- **Solves:** the deliverable to outside counsel / leadership.
- **Does not:** export to PDF (that's a SaaS-premium feature).

10.13 Audit

The screenshot displays the 'Audit' section of the forenix-oss application. The left sidebar contains navigation links for OSINT, PIPELINE, and FORENSICS. The main panel shows the 'Audit log' with a description: 'Append-only chain across both workflows. Every write computes sha256(prevHash | action | entity | entityId | iso(t)).' A search bar is present with the text 'action, entity, hash...'. Below the search bar is a table of 21 entries.

HASH	ACTION	ENTITY	SCOPE	WHEN
b84a5d97...58b8 ← 00000000...0000	create_user	User cnp7lagh1000	—	8m ago
b405962f...13b7 ← b84a5d97...58b8	create_user	User cnp7laghv000	—	8m ago
02469c9c...2e6e ← b405962f...13b7	create_user	User cnp7laghz000	—	7m ago
0bae3843...57a2 ← 02469c9c...2e6e	create_investigation	Investigation cnp7lagk5001	Inv	7m ago
1e80e7a4...3fa0 ← 0bae3843...57a2	run_pipeline	Investigation cnp7lagk5001	Inv	7m ago
c0893b89...0860 ← 1e80e7a4...3fa0	create_case	Case cnp7lag13000	case	7m ago
72715b02...1df8 ← c0893b89...0860	promote_to_evidence	Finding f-1	case	7m ago
0996a640...2dcc ← 72715b02...1df8	verify_evidence	EvidenceCommit cnp7lagig000	case	7m ago
3de0414a...cace ← 0996a640...2dcc	generate_report	Report r-1	Inv	7m ago
023e80f8...c7db ← 3de0414a...cace	pipeline_started	Investigation cnp7lagkb001	Inv	5m ago
3dfc7f00...a1be ← 023e80f8...c7db	agent_identity_completed	Investigation cnp7lagkb001	Inv	5m ago
a5034c70...abe7 ← 3dfc7f00...a1be	agent_social_completed	Investigation cnp7lagkb001	Inv	5m ago
9ef2a8b2...a64f ← a5034c70...abe7	agent_geo_completed	Investigation cnp7lagkb001	Inv	5m ago
750f7d60...18ed ← 9ef2a8b2...a64f	entities_extracted	Investigation cnp7lagkb001	Inv	4m ago
0aa9d23...f239	report_generated	Report	Inv	4m ago

- **What:** the full hash-chained log.
- **Solves:** "who changed what, when, in what order."
- **Does not:** allow direct writes (the chain is computed by the system; you can't insert a row out-of-band).

10.14 Integrity

The screenshot displays the 'Integrity' section of the forenix-oss application. The left sidebar contains a navigation menu with categories: OSINT (Dashboard, Investigations, Entity Graph, Monitors, Verification, Reports), PIPELINE (Pipeline), and FORENSICS (Cases, Evidence, Network Graph, Branch Graph, AI Lab, Integrity, Audit, Reviews). The 'Integrity' item is highlighted. The main content area shows the 'Integrity' title and a subtitle: 'Replay the entire audit log in insertion order and verify every row's SHA-256 against the recomputed value.' A 'Verify chain' button is present. A green checkmark icon indicates 'CHAIN VERIFIED' with the message '21 entries — every hash recomputes to the stored value.' Below this, a 'METHOD' section shows a code snippet for a linear replay function. A detailed description explains that the process walks the table in 'createdAt' order, recomputing each row's hash from the previous row's hash and stopping at the first mismatch. It notes that 'GENESIS_HASH' is 32 zero bytes, hex-encoded, and that the walk is cheap due to a single 'findMany' and a CPU loop.

Integrity FORENIX-OSS adapter mock v010 online

Integrity
Replay the entire audit log in insertion order and verify every row's SHA-256 against the recomputed value. Verify chain

CHAIN VERIFIED
21 entries — every hash recomputes to the stored value.

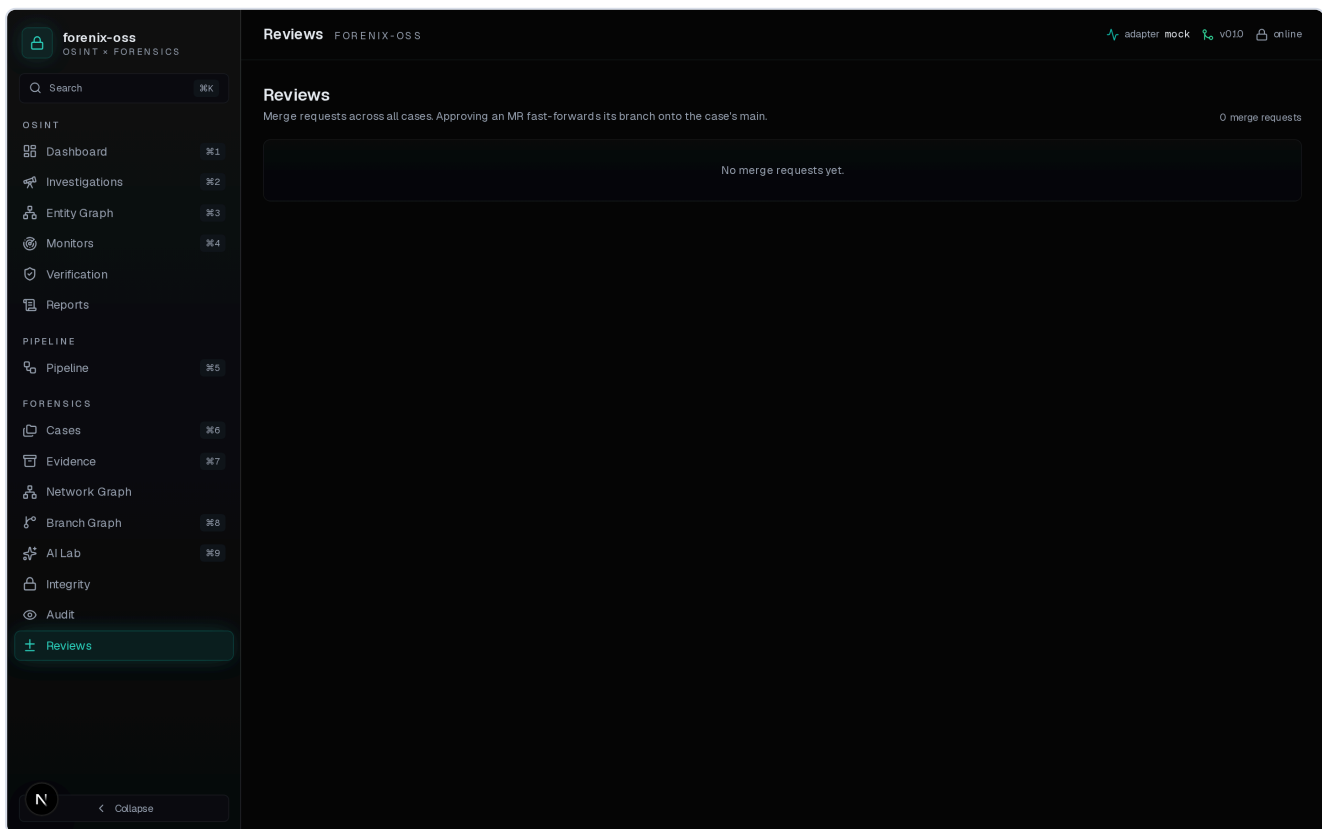
METHOD

```
hash(row_n) = sha256(
  hash(row_{n-1})
  | action
  | entity
  | entityId
  | iso(createdAt)
)
```

Replay walks the table in createdAt order, recomputes each row's hash from the previous row's hash, and stops at the first mismatch. GENESIS_HASH is 32 zero bytes, hex-encoded. The walk is cheap — a single findMany + a CPU loop.

- **What:** one-button replay of the entire chain.
- **Solves:** proving tamper-evidence.
- **Does not:** use Merkle batching yet — linear replay is fast enough for sub-10M-row chains.

10.15 Reviews



- **What:** list of merge requests across all cases.
- **Solves:** second-pair-of-eyes review before evidence is sealed.
- **Does not:** offer inline merge/approve buttons yet — that ships in Phase 4 polish.

11. Working with the AI adapters

forenix-oss ships with six AI adapters. You can switch between them by editing one line in `.env` :

```
AI_ADAPTER=mock           # default – deterministic seeded output
# AI_ADAPTER=ollama       # local Ollama HTTP API
# AI_ADAPTER=glm          # Zhipu AI GLM-4 / GLM-5
# AI_ADAPTER=claude       # Anthropic Claude (SaaS-gated)
# AI_ADAPTER=openrouter   # OpenRouter proxy → many models
# AI_ADAPTER=nvidia       # NVIDIA NIM hosted catalogue
```

After changing the value, restart the dev server. The top-bar adapter chip will reflect the change.

11.1 Per-call adapter override

You don't always want to commit to one adapter. Pass `adapter` in the pipeline-run body to override for a single run:

```
curl -X POST -H "content-type: application/json" \
  -d '{"agentGroups":["identity","social"],"adapter":"openrouter"}' \
  http://localhost:3000/api/pipeline/run/<INVESTIGATION_ID>
```

Use this when you want to A/B two models on the same target — say, NVIDIA's Llama-3.3-70B vs OpenRouter's `gpt-oss-120b:free` — and compare findings side-by-side.

11.2 Picking an adapter

Choose this	When
mock	demos, dev, screenshots, tests
ollama	air-gapped deployments / local-only workflows
openrouter	the most flexibility — one key, dozens of models
nvidia	high-quality 70B models on a free dev tier
glm	sovereign-China deployments / GLM-4 quality preference
claude	SaaS premium tier only

11.3 Costs + latency

Adapter	Cost (this build)	Latency (3-group run)
mock	\$0	< 1 s
nvidia	free dev tier	≈ 45-60 s
openrouter (gpt-oss-120b:free)	\$0	≈ 80 s
openrouter (paid models)	\$0.20 - \$5 per 1M tokens	15-30 s

The audit chain stays valid identically regardless of which adapter you use. The adapter swap only affects the *quality* and *latency* of the findings.

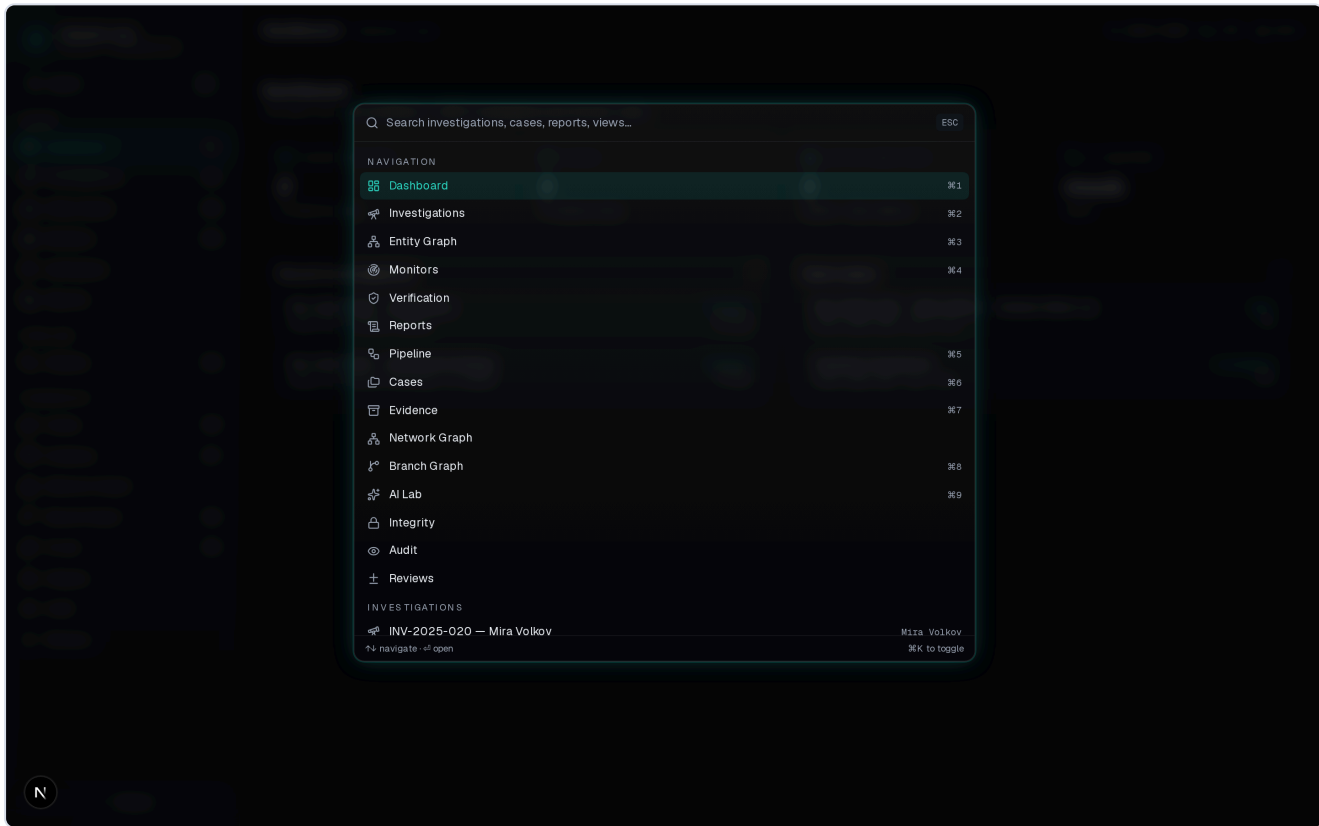
12. Keyboard shortcuts + command palette

12.1 View shortcuts

Shortcut	View
⌘1 / Ctrl+1	Dashboard
⌘2 / Ctrl+2	Investigations
⌘3 / Ctrl+3	Entity Graph
⌘4 / Ctrl+4	Monitors
⌘5 / Ctrl+5	Pipeline
⌘6 / Ctrl+6	Cases
⌘7 / Ctrl+7	Evidence
⌘8 / Ctrl+8	Branch Graph
⌘9 / Ctrl+9	AI Lab

12.2 The command palette

⌘K / Ctrl+K opens the palette from anywhere.



- Type to filter across nav items, investigations, cases, and reports.
- ↑ / ↓ to navigate.

- ↵ **Enter** to open.
- **Esc** to close.

The palette is the fastest way to jump straight to a specific case or investigation by number/title.

12.3 Other useful shortcuts

- **Click any audit row's investigation/case scope chip** — switches view + activates the right detail panel in one click.
- **Click the bridge chip on an investigation** — opens its linked case.
- **Click a finding's *promote* → *evidence* button** — does the promotion + writes the audit row + invalidates the queries that feed the surrounding UI.

13. Tips for new analysts

13.1 The verification habit

Make a habit of opening the Audit view at the end of every session and confirming `verifyAuditChain()` is still `{ ok: true }`. The check takes < 1 second and gives you the single most important guarantee the platform offers.

13.2 Promote sparingly

Only promote findings you genuinely intend to bring under chain of custody. A promoted finding becomes Evidence with a hash that will appear on every audit + court export. Use the verify-but- don't-promote pattern when you want to upgrade confidence but keep the item out of the forensic side.

13.3 Branch instead of overwriting

If you need to refine an evidence row's metadata, never edit it in-place. Create a new branch on the case, make the change there, open a Merge Request, and let a second pair of eyes review. This is the same workflow as Git, and for the same reason.

13.4 Filter before scrolling

Every list view supports an inline filter. Use it. The platform scales to thousands of investigations and tens of thousands of findings — scrolling does not.

13.5 Trust the chain

When the Integrity view turns green, it is genuinely green. The chain is cheap to verify (linear pass over a single table) and trivial to reproduce offline. Don't accept "trust us" answers from anyone — including us.

14. Troubleshooting

14.1 The dev server doesn't start

```
bun install      # re-install deps
bun run db:push  # re-apply the schema
bun run db:seed  # re-seed
bun run dev
```

If `bun` isn't installed, you can use `node` directly — `next dev` works the same. The seed script requires `tsx`, which is already in `devDependencies`.

14.2 The Audit Integrity view turns red

The chain has been mutated outside the platform. Do **not** try to "fix" it — the integrity guarantee depends on this being impossible to fix invisibly. Instead:

1. Note the `brokenAt` row id and timestamp.
2. Check `prisma/dev.db` (SQLite) or your Postgres for direct writes around that timestamp.
3. Escalate per your incident-response policy.

The chain remains broken until the next legitimate write — at which point the `brokenAt` row's `prevHash` is no longer the "tip" of the chain, but it remains visible to all future verifies.

14.3 A pipeline run hangs

Most hangs are slow upstream models. The platform aborts any single adapter call at 90 seconds.

- Switch to a faster model: `OPENROUTER_MODEL=openai/gpt-oss-120b:free`.
- Drop agent groups: try `identity` only first.
- Check the dev log: `tail -f /tmp/forenix-live.log` (or whatever you set as your log path).

14.4 The Pipeline runner shows "no investigations"

You haven't seeded the database. Run:

```
bun run db:seed
```

...or create one with the **+ New** button on the Investigations view.

14.5 "Pipeline 500" right after the run

The platform completed the run but failed to serialize the response (we've fixed every `BigInt` path in 0.1, but a future field might trip this again). The chain is still valid — refresh the investigation detail to see the findings landed.

15. Frequently asked questions

Does forenix-oss store any data outside my own database?

No. The platform is single-tenant. The only outbound traffic is to the configured AI adapter's API endpoint (you control that via `.env`).

Can I use a locally-hosted LLM?

Yes — the `ollama` adapter is built for exactly that. Pull a model with `ollama pull qwen2.5:7b-instruct`, set `AI_ADAPTER=ollama`, restart the dev server.

Can a non-technical team member use this?

Yes — the entire daily workflow is in the web UI. The only thing you need a terminal for is the initial install + the seed. After that, point your browser at `localhost:3000` and never look back.

How big can the audit chain get?

Tested cleanly to 100K rows on a single SQLite file. Beyond that, Postgres + the same algorithm gets you to tens of millions. We'll add Merkle batching when our first user hits the wall.

Will my findings stay private?

If you use a hosted adapter (NVIDIA / OpenRouter / Claude / GLM), the target name + search-result snippets are sent to that provider for analysis. Read each provider's data-handling policy. For zero outbound traffic, use `ollama` or `mock`.

Can I cite a forenix-oss case in a court filing?

The chain-of-custody methodology is documented and reproducible (see `docs/07-SECURITY.md`). Whether a court accepts it is a local matter — we'd love to hear from any user who runs this through their first admissibility argument.

16. Glossary

Term	Meaning
Adapter	A concrete implementation of the AI interface — <code>mock</code> , <code>ollama</code> , <code>glm</code> , <code>claude</code> , <code>openrouter</code> , <code>nvidia</code> . Pluggable, swappable with one env var.
Agent group	A bundle of OSINT agents that share a focus (e.g. <code>identity</code> , <code>infrastructure</code>). The pipeline runs the agents inside a group in parallel.
Audit chain	The SHA-256 forward chain over the <code>AuditLog</code> table. The trust anchor of the platform.
Branch	A named line of evidence-commit history under a case. Each case has a <code>main</code> branch by default.
Bridge	The act of linking an Investigation to a Case (via <code>Investigation.caseId</code>) and optionally promoting findings to Evidence.
Case	A forensic workspace. Owns Evidence, Branches, MRs, Assignments.
Commit	A versioned change to a piece of Evidence. Carries a SHA-256, a parent hash, an author, and a change-type.
Confidence	A finding's likelihood label: <code>confirmed</code> , <code>probable</code> , <code>unverified</code> , <code>disputed</code> , <code>false</code> .
Entity	A normalised actor in the OSINT graph — person, org, domain, IP, email, phone, account, location.
Evidence	A row in the forensic side of the schema. Carries a content hash + commit history. Can be <code>collected</code> / <code>verified</code> / <code>sealed</code> .
Finding	A discrete OSINT signal produced by an agent. Lives on the Investigation; can be promoted to Evidence.
Investigation	The OSINT collection workspace, bound to a <i>target</i> .
Merge Request	A request to merge one Branch into another, gated by review.
Monitor	A cadenced re-run schedule for an Investigation.
Pipeline	The end-to-end run that orchestrates <code>analyzePipeline</code> → <code>extractEntities</code> → <code>generateReport</code> over the AI adapter.
Promote	The act of turning a Finding into an Evidence row (one click in the UI).
Reasoning trace	The agent's free-text explanation of why it produced a finding. Stored on the Finding.
Report	A structured deliverable. Sectioned JSON (Argus-side) or markdown (ForenX-side).
Seal	Mark an Evidence row immutable. Writes a <code>seal</code> commit + audit row.
Target	The person / org / domain / IP / etc. an Investigation is about.
Verify (chain)	Replay every audit row's hash and confirm none have been mutated.

Term	Meaning
Verify (finding)	Flip a Finding's <code>verified</code> flag to <code>true</code> and record the verifier.

End of manual. For engineering / deployment / architecture references, see the documents under `docs/`.

If you spot something incorrect or unclear, please open an issue at <https://github.com/thunderstornX/forenix-oss/issues>.